



حل پروژه‌ی طبقه‌بندی از کلاس یادگیری ماشین

گردآورنده: امیرحسین ترنجیان

کلاس: یادگیری ماشین

تابستان ۱۴۰۵

فهرست

۵.....	معرفی مجموعه داده
۹.....	بررسی و رفع مقادیر گم‌شده
۱۳.....	هدف بررسی مقادیر اشتباه
۱۳.....	مفهوم تئوری مقادیر اشتباه
۱۵.....	نمایش و حذف داده‌های پرت
۱۶.....	مفهوم Boxplot
۱۷.....	حذف مقادیر پرت از ستون Ph
۱۸.....	حذف مقادیر پرت از ستون Hardness
۱۹.....	حذف مقادیر پرت از ستون Solids
۲۰.....	حذف مقادیر پرت از ستون Chloramines
۲۱.....	حذف مقادیر پرت از ستون Sulfate
۲۲.....	حذف مقادیر پرت از ستون Organic_cabon
۲۳.....	حذف مقادیر پرت از ستون Trihalometanes
۲۳.....	مزایای حذف داده‌های پرت:
۲۴.....	تبدیل داده‌های Object به داده‌های عددی
۲۵.....	توصیف خروجی
۲۶.....	خوشه‌بندی داده‌ها (Clustering)
۲۹.....	تحلیل همبستگی ویژگی‌ها
۲۹.....	نمایش همبستگی با Heatmap
۳۰.....	نمایش همبستگی با Pairplot

۳۱	تعیین ورودی و خروجی
۳۱	استانداردسازی (Standardization) و نرمال سازی (Normalization)
۳۲	• نرمال سازی (Normalization)
۳۲	• استانداردسازی (Standardization)
۳۳	دلیل استفاده از StandardScaler در این پروژه
۳۴	مدل های طبقه بندی
۳۴	Logistic Regression
۳۵	فرمول تابع سیگموئید
۳۶	K-Nearest Neighbors (KNN)
۳۹	Support Vector Machine (SVC)
۳۹	فرمول ابرصفحه تصمیم
۳۹	فرمول کرنل RBF
۴۱	Random Forest
۴۲	فرمول رأی گیری نهایی
۴۴	AdaBoost
۴۸	Gradient Boosting
۴۹	نقش Learning Rate
۴۹	پارامترهای مهم Gradient Boosting
۴۹	n_estimators
۵۰	max_depth
۵۰	subsample
۵۲	تنظیم پارامترها با GridSearchCV
۵۳	بررسی رابطه بین دو ویژگی با استفاده از Clustering
۵۴	■ فاصله اقلیدسی (Euclidean Distance)

۵۵.....	محاسبه مرکز خوشه (Centroid)	■
۵۸.....	ارزیابی مدل ها	
۵۹.....	Accuracy	
۵۹.....	Precision	
۵۹.....	Recall	
۵۹.....	F1-Score	
۶۰.....	Confusion Matrix	
۶۰.....	Confusion Matrix Display	
۶۱.....	Classification Report	
۶۱.....	نتایج دقت	

پروژه طبقه‌بندی کیفیت آب آشامیدنی با استفاده از الگوریتم‌های یادگیری ماشین

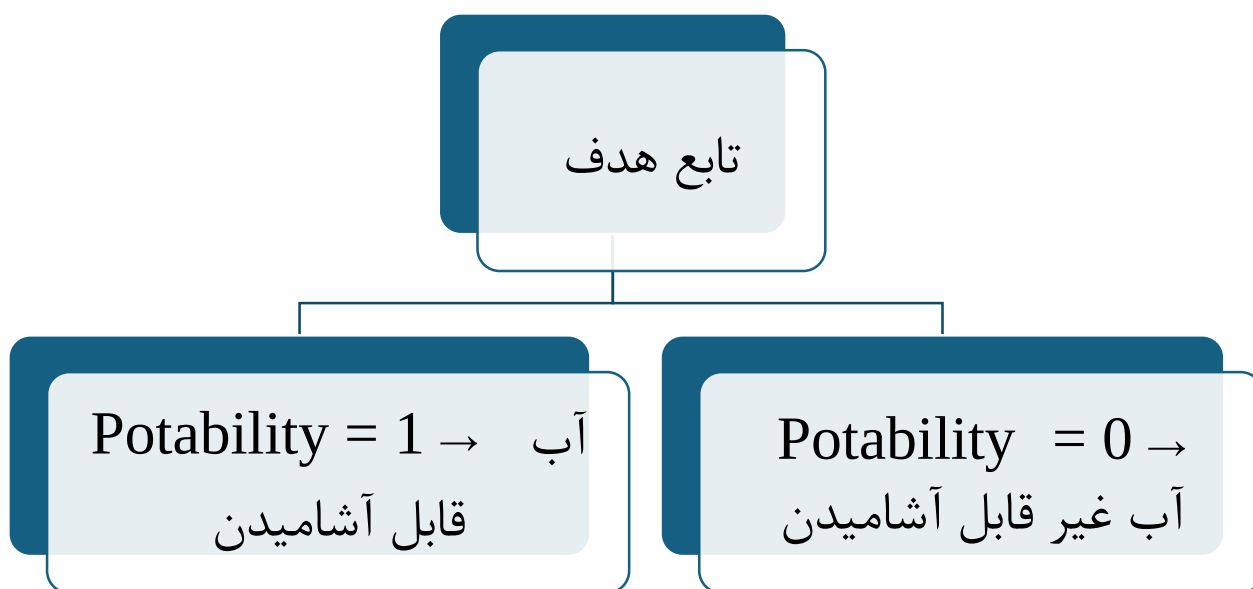
دسترسی به آب آشامیدنی سالم یکی از مهم‌ترین نیازهای جوامع انسانی است. ارزیابی کیفیت آب به روش‌های آزمایشگاهی معمولاً هزینه‌بر و زمان‌بر است. در سال‌های اخیر روش‌های یادگیری ماشین به عنوان ابزاری قدرتمند برای پیش‌بینی کیفیت آب مورد توجه قرار گرفته‌اند. هدف این پروژه، پیش‌بینی قابلیت آشامیدن آب (Potability) بر اساس ویژگی‌های فیزیکی و شیمیایی آب است.

معرفی مجموعه داده

در این پروژه از مجموعه داده Water Quality استفاده شده است. این مجموعه داده شامل تعدادی ویژگی فیزیکی و شیمیایی آب بوده و متغیر هدف آن Potability است. ویژگی‌های موجود در دیتاست عبارت‌اند از:

pH (میزان اسیدی بودن آب)
Hardness (سختی آب)
Solids (کل مواد جامد محلول)
Chloramines (کلرامین‌ها)
Sulfate (سولفات)
Conductivity (هدایت الکتریکی یا رسانایی)
Organic Carbon (کربن آلی)
Trihalomethanes (تری‌هالومتان‌ها)

متغیر هدف:



در اولین مرحله، مجموعه داده موردنظر با استفاده از کتابخانه Pandas در محیط پایتون بارگذاری شد. این مجموعه داده شامل اطلاعات مربوط به دانش آموزان از جنبه‌های مختلف فردی، خانوادگی، اجتماعی و آموزشی است. هدف از این مرحله آشنایی اولیه با ساختار داده‌ها، تعداد ویژگی‌ها و نوع اطلاعات موجود در هر ستون می‌باشد. همچنین با نمایش چند سطر ابتدایی داده‌ها، صحت بارگذاری فایل بررسی شد.

در اولین مرحله، داده‌های پروژه از فایل CSV خوانده شدند تا بتوان عملیات تحلیل و مدل‌سازی را روی آن انجام داد.

کد

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ 9.9s

توضیح کد

`import numpy as np`

کتابخانه Numpy برای انجام محاسبات عددی و کار با آرایه‌های چندبعدی استفاده می‌شود.

`import pandas as pd`

کتابخانه Pandas برای خواندن، ذخیره‌سازی و پردازش داده‌های جدولی استفاده می‌شود.

`import matplotlib.pyplot as plt`

این کتابخانه برای رسم نمودارها و نمایش نتایج گرافیکی استفاده می‌شود.

`import seaborn as sns`

کتابخانه Seaborn برای رسم نمودارهای آماری و تحلیل داده‌ها استفاده می‌شود.

هدف

اولین مرحله در هر پروژه یادگیری ماشین، بارگذاری و بررسی اولیه داده‌ها است. در این مرحله داده‌ها از فایل CSV خوانده شده و ساختار کلی آن‌ها بررسی می‌شود.

تحلیل دیتاست پروژه

داده‌ها مهم‌ترین بخش هر مدل یادگیری ماشین هستند. کیفیت داده مستقیماً روی عملکرد مدل تأثیر می‌گذارد. بنابراین قبل از هرگونه مدل‌سازی باید نوع ستون‌ها، تعداد رکوردها و وجود داده‌های ناقص بررسی شود.

کد:

```
df=pd.read_csv("C:/Intel/water_quality.csv")
df.head(15)
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
0	NaN	204.890455	20791.318981	7.300212	368.516441	564.308654	10.379783	86.990970	2.963135	0
1	3.716080	129.422921	18630.057858	6.635246	NaN	592.885359	15.180013	56.329076	4.500656	0
2	8.099124	224.236259	19909.541732	9.275884	NaN	418.606213	16.868637	66.420093	3.055934	0
3	8.316766	214.373394	22018.417441	8.059332	356.886136	363.266516	18.436524	100.341674	4.628771	0
4	9.092223	181.101509	17978.986339	6.546600	310.135738	398.410813	11.558279	31.997993	4.075075	0
5	5.584087	188.313324	28748.687739	7.544869	326.678363	280.467916	8.399735	54.917862	2.559708	0
6	10.223862	248.071735	28749.716544	7.513408	393.663396	283.651634	13.789695	84.603556	2.672989	0
7	8.635849	203.361523	13672.091764	4.563009	303.309771	474.607645	12.363817	62.798309	4.401425	0
8	NaN	118.988579	14285.583854	7.804174	268.646941	389.375566	12.706049	53.928846	3.595017	0
9	11.180284	227.231469	25484.508491	9.077200	404.041635	563.885481	17.927806	71.976601	4.370562	0
10	7.360640	165.520797	32452.614409	7.550701	326.624353	425.383419	15.586810	78.740016	3.662292	0
11	7.974522	218.693300	18767.656682	8.110385	NaN	364.098230	14.525746	76.485911	4.011718	0
12	7.119824	156.704993	18730.813653	3.606036	282.344050	347.715027	15.929536	79.500778	3.445756	0
13	NaN	150.174923	27331.361962	6.838223	299.415781	379.761835	19.370807	76.509996	4.413974	0
14	7.496232	205.344982	28388.004887	5.072558	NaN	444.645352	13.228311	70.300213	4.777382	0

تحلیل

در این مرحله تعداد نمونه‌ها و ویژگی‌های موجود در دیتاست بررسی شد.

```
df.shape
```

✓ 0.0s

(3276, 10)

➤ تعداد ردیف‌ها: ۳۲۷۶

ویژگی‌ها: ۱۰

ابتدا ساختار داده‌ها با استفاده از توابع زیر مورد بررسی قرار گرفت:

```
df.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3276 entries, 0 to 3275
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   ph                    2785 non-null   float64
1   Hardness              3276 non-null   float64
2   Solids                3276 non-null   float64
3   Chloramines           3276 non-null   float64
4   Sulfate               2495 non-null   float64
5   Conductivity          3276 non-null   float64
6   Organic_carbon        3276 non-null   float64
7   Trihalomethanes       3114 non-null   float64
8   Turbidity             3276 non-null   float64
9   Potability            3276 non-null   int64
```


- تعداد رکوردها
- نوع داده هر ستون
- تعداد مقادیر غیر Null

```
df.describe()
```

✓ 0.1s

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
count	2785.000000	3276.000000	3276.000000	3276.000000	2495.000000	3276.000000	3276.000000	3114.000000	3276.000000	3276.000000
mean	7.080795	196.369496	22014.092526	7.122277	333.775777	426.205111	14.284970	66.396293	3.966786	0.390110
std	1.594320	32.879761	8768.570828	1.583085	41.416840	80.824064	3.308162	16.175008	0.780382	0.487849
min	0.000000	47.432000	320.942611	0.352000	129.000000	181.483754	2.200000	0.738000	1.450000	0.000000
25%	6.093092	176.850538	15666.690297	6.127421	307.699498	365.734414	12.065801	55.844536	3.439711	0.000000
50%	7.036752	196.967627	20927.833607	7.130299	333.073546	421.884968	14.218338	66.622485	3.955028	0.000000
75%	8.062066	216.667456	27332.762127	8.114887	359.950170	481.792304	16.557652	77.337473	4.500320	1.000000
max	14.000000	323.124000	61227.196008	13.127000	481.030642	753.342620	28.300000	124.000000	6.739000	1.000000

در این مرحله تعداد هر ستون ، میانگین مقادیر ، انحراف معیار، کمترین مقدار ، بیشترین مقدار موجود در هر ستون ، و مقادیری که از ۲۵ درصد ، ۵۰ درصد و ۷۵ درصد اعداد موجود در ستون ها بزرگ تر است نشان داده می شود. با بررسی نتایج مشخص شد که ویژگی های مختلف دارای بازه های عددی متفاوتی هستند؛ بنابراین در مراحل بعدی استانداردسازی داده ها ضروری خواهد بود.

بررسی و رفع مقادیر گم شده

وجود مقادیر گم شده یکی از مشکلات رایج در داده های واقعی است .ابتدا تعداد مقادیر گم شده

هر ستون بررسی شد:

```
df.isnull().sum()
```

✓ 0.0s

ph	491
Hardness	0
Solids	0
Chloramines	0
Sulfate	781
Conductivity	0
Organic_carbon	0
Trihalomethanes	162
Turbidity	0

همانگونه مشاهده می‌فرمایید در سه ستون مقادیر گمشده وجود دارند که عبارتند از:

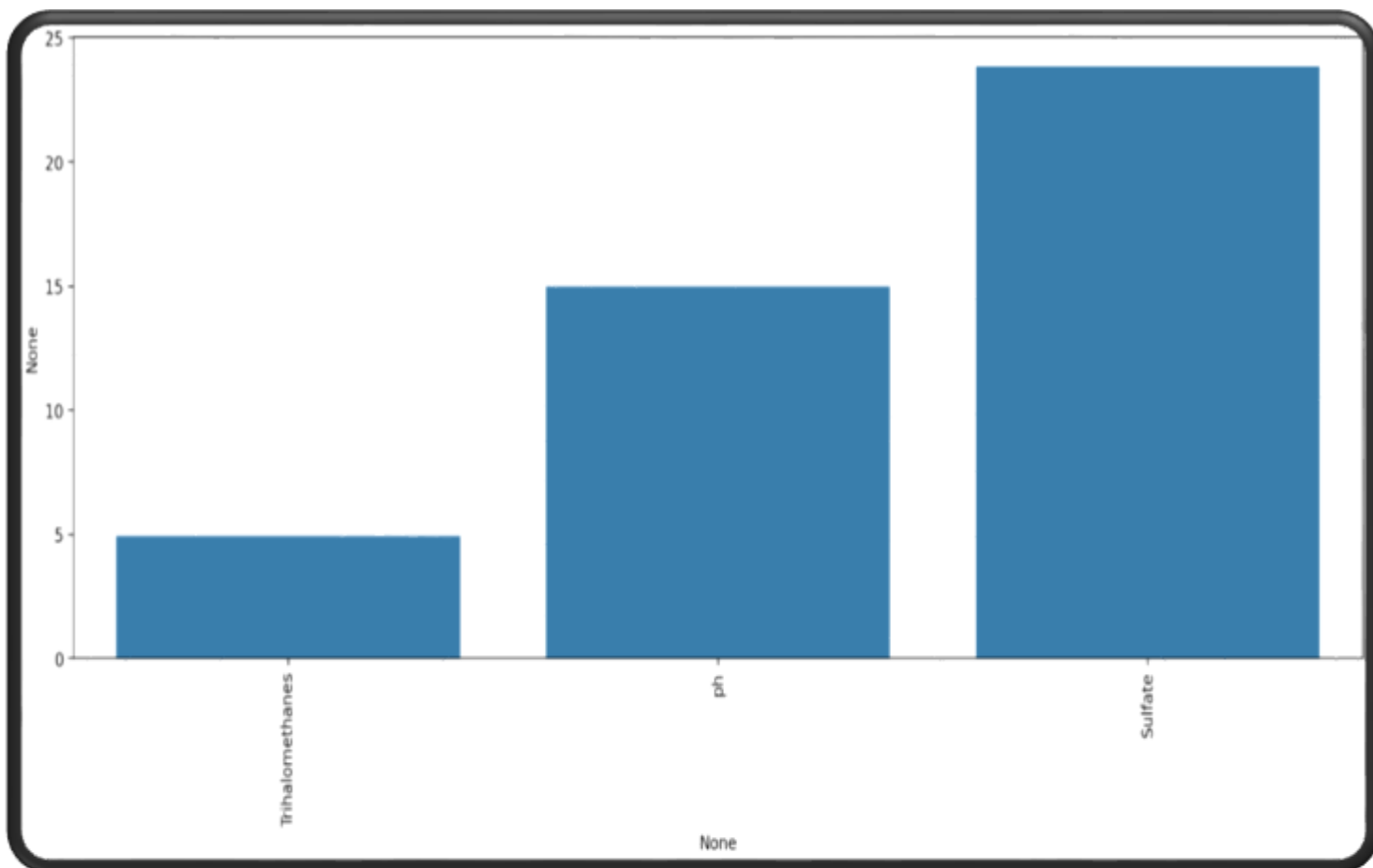
- Ph
- Sulfate
- Trihalomethanes

سپس درصد داده‌های گمشده برای هر ویژگی محاسبه و نمایش داده شد.

```
def calculate_missing_percent(df):  
    nan_percent = 100 * df.isnull().sum() / len(df)  
    nan_percent = nan_percent[nan_percent>0].sort_values()  
  
    return nan_percent  
  
✓ 0.0s  
  
nan_percent = calculate_missing_percent(df)  
  
✓ 0.0s  
  
nan_percent  
✓ 0.0s  
  
Trihalomethanes    4.945055  
ph                 14.987790  
Sulfate            23.840049  
dtype: float64
```

همانگونه که مشاهده می‌فرمایید در ستون ph تقریباً ۵ درصد کل مقادیر، گمشده هستند. در ستون Trihalomethanes ۱۵ درصد کل مقادیر ستون، گمشده هستند. در ستون Sulfate ۲۴ درصد کل مقادیر ستون، گمشده هستند. برای نمایش درصد داده‌های گمشده از نمودار ستونی استفاده شد.

```
plt.figure(figsize=(16,6), dpi=200)  
sns.barplot(x=nan_percent.index,y=nan_percent)  
plt.xticks(rotation=90)  
plt.ylim()  
  
✓ 0.9s
```



در گام بعدی، به سراغ بررسی همبستگی ستون های دارای مقادیر گم شده با سایر ستون ها می رویم. این به این جهت است که می خواهیم مقادیر گم شده را بر اساس میانگین ستون های دیگر جایگزین کنیم.

```
df.corr()['ph'].sort_values(ascending=False)
✓ 0.0s
```

Variable	Correlation with ph
ph	1.000000
Hardness	0.082096
Organic_carbon	0.043503
Conductivity	0.018614
Sulfate	0.018203
Trihalomethanes	0.003354
Potability	-0.003556
Chloramines	-0.034350
Turbidity	-0.039057
Solids	-0.089288

Name: ph, dtype: float64

```
df.corr()['Sulfate'].sort_values(ascending=False)
✓ 0.0s
```

Sulfate	1.000000
Organic_carbon	0.030831
Chloramines	0.027244
ph	0.018203
Turbidity	-0.011187
Conductivity	-0.016121
Potability	-0.023577
Trihalomethanes	-0.030274
Hardness	-0.106923
Solids	-0.171804

Name: Sulfate, dtype: float64

```
df.corr()['Trihalomethanes'].sort_values(ascending=False)
✓ 0.0s
```

Trihalomethanes	1.000000
Chloramines	0.017084
Potability	0.007130
ph	0.003354
Conductivity	0.001285
Solids	-0.009143
Hardness	-0.013013
Organic_carbon	-0.013274
Turbidity	-0.022145
Sulfate	-0.030274

Name: Trihalomethanes, dtype: float64

همانگونه که مشاهده می‌فرمایید ارتباط ستون‌های دارای مقادیر گم‌شده با سایر ستون‌ها کم است و خیلی برای جایگزینی با میانگین سایر ستون‌ها مناسب نیستند.

پس از بررسی همبستگی ویژگی‌ها، مقادیر گم‌شده هر ستون بر اساس میانگین گروه‌های Potability جایگزین شدند:

```
df['ph'] = df.groupby('Potability')['ph'].transform(
    lambda x: x.fillna(x.mean())
)

df['Sulfate'] = df.groupby('Potability')['Sulfate'].transform(
    lambda x: x.fillna(x.mean())
)

df['Trihalomethanes'] = df.groupby('Potability')['Trihalomethanes'].transform(
    lambda x: x.fillna(x.mean())
)
✓ 0.0s
```

همین روش برای ویژگی‌های Sulfate و Trihalomethanes نیز اعمال شد.

دلیل انتخاب این روش آن است که میانگین هر کلاس اطلاعات دقیق‌تری نسبت به میانگین کل دیتاست ارائه می‌دهد و ساختار داده‌ها بهتر حفظ می‌شود.

```
df.isnull().sum()
✓ 0.0s
ph          0
Hardness    0
Solids       0
Chloramines  0
Sulfate      0
Conductivity 0
Organic_carbon 0
Trihalomethanes 0
Turbidity    0
Potability   0
dtype: int64
```

بررسی مقادیر اشتباه

هدف بررسی مقادیر اشتباه

شناسایی داده‌های غیرمنطقی و پرت.

مفهوم تئوری مقادیر اشتباه

گاهی داده‌ها از نظر آماری معتبر هستند اما از نظر منطقی اشتباه‌اند.

علاوه بر مقادیر گم‌شده و داده‌های پرت، یکی دیگر از مشکلات رایج در مجموعه داده‌ها وجود مقادیر نامعتبر (Wrong Values) است. این مقادیر ممکن است در اثر خطای اندازه‌گیری، ورود اشتباه داده‌ها یا نقص در فرآیند جمع‌آوری داده ایجاد شوند.

مقادیر نامعتبر معمولاً شامل موارد زیر هستند:

- مقادیر خارج از محدوده منطقی
- اعداد منفی در ویژگی‌هایی که نباید منفی باشند
- مقادیر غیرواقعی و غیرممکن
- داده‌های ناسازگار با ماهیت مسئله

در این پروژه ابتدا بازه مقادیر هر ویژگی با استفاده از آمار توصیفی بررسی شد:

```
for col in df.columns:
    print(col)
    print("Min =",df[col].min())
    print("Max =",df[col].max())
```

✓ 0.1s

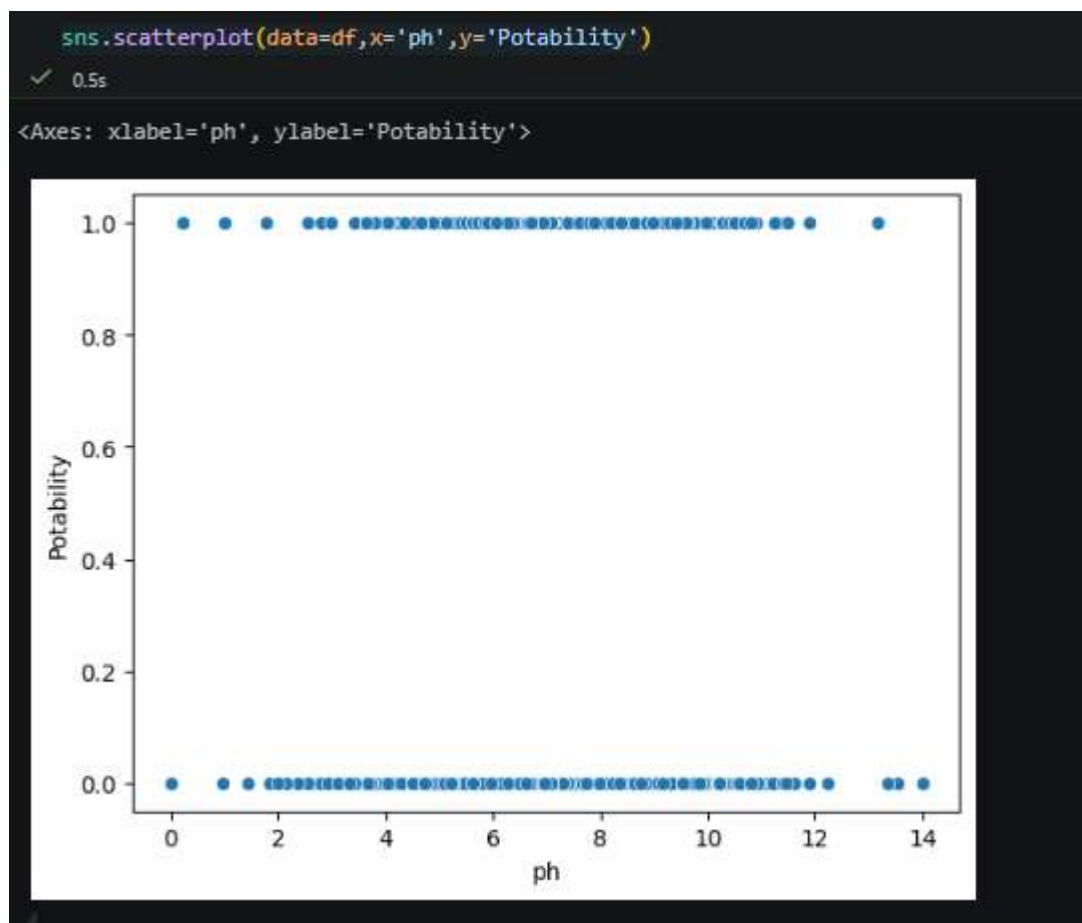
```
ph
Min = 1.4317815547427415
Max = 12.246928068519756
Hardness
Min = 73.4922336890611
Max = 304.23591211231667
Solids
Min = 320.942611274359
Max = 56351.39630372492
Chloramines
Min = 1.3908709048851806
Max = 13.127000000000002
Sulfate
Min = 180.20674636482343
Max = 481.0306423059972
Conductivity
Min = 181.483753985146
Max = 695.3695279880473
Organic_carbon
Min = 4.371898608441143
Max = 28.300000000000001
Trihalomethanes
Min = 8.175876384274268
Max = 124.0
Turbidity
...
Max = 6.739
Potability
Min = 0
Max = 1
```

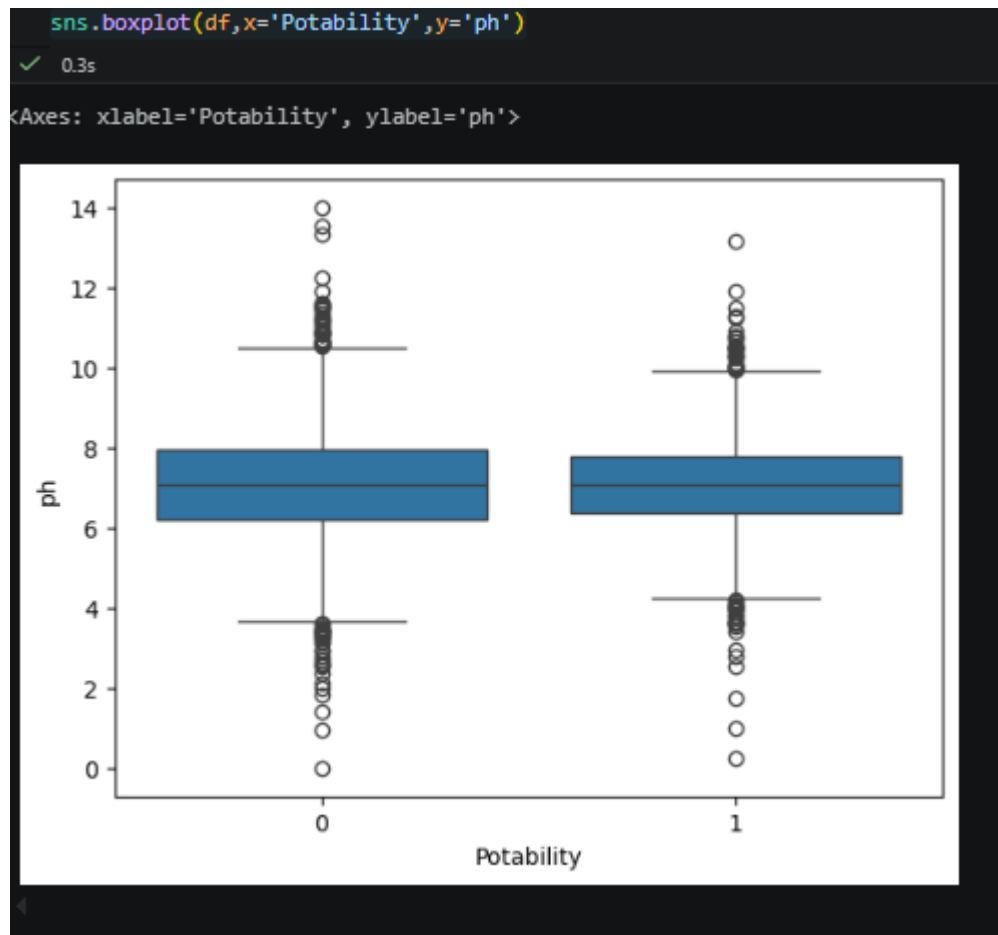
به عنوان مثال:

- ❖ مقدار PH معمولاً باید در بازه ۰ تا ۱۴ قرار داشته باشد .
- ❖ سختی آب (Hardness) نمی تواند منفی باشد .
- ❖ هدایت الکتریکی (Conductivity) باید مقداری مثبت داشته باشد .
- ❖ کدورت آب (Turbidity) نمی تواند مقدار منفی داشته باشد.

نمایش و حذف داده های پرت

برای پیدا کردن Outliers از Scatterplot و Boxplot استفاده کردیم.





مفهوم Boxplot

Boxplot ابزاری برای شناسایی Outlier است.

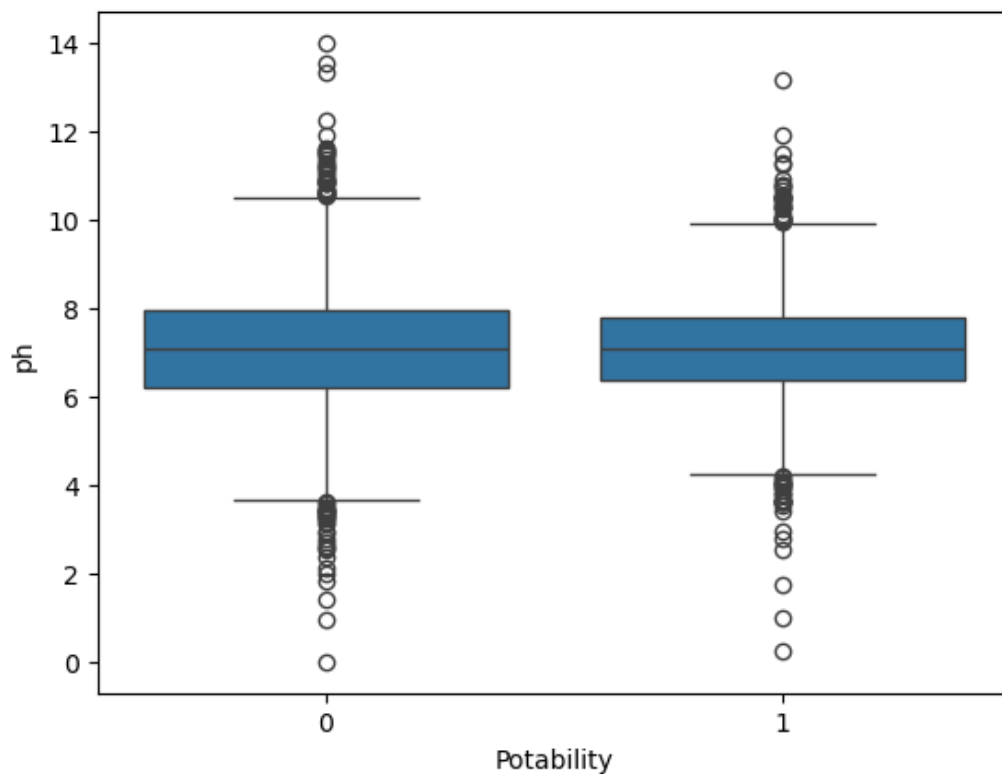
اجزای آن:

- Median
- Quartile اول
- Quartile سوم
- نقاط پرت

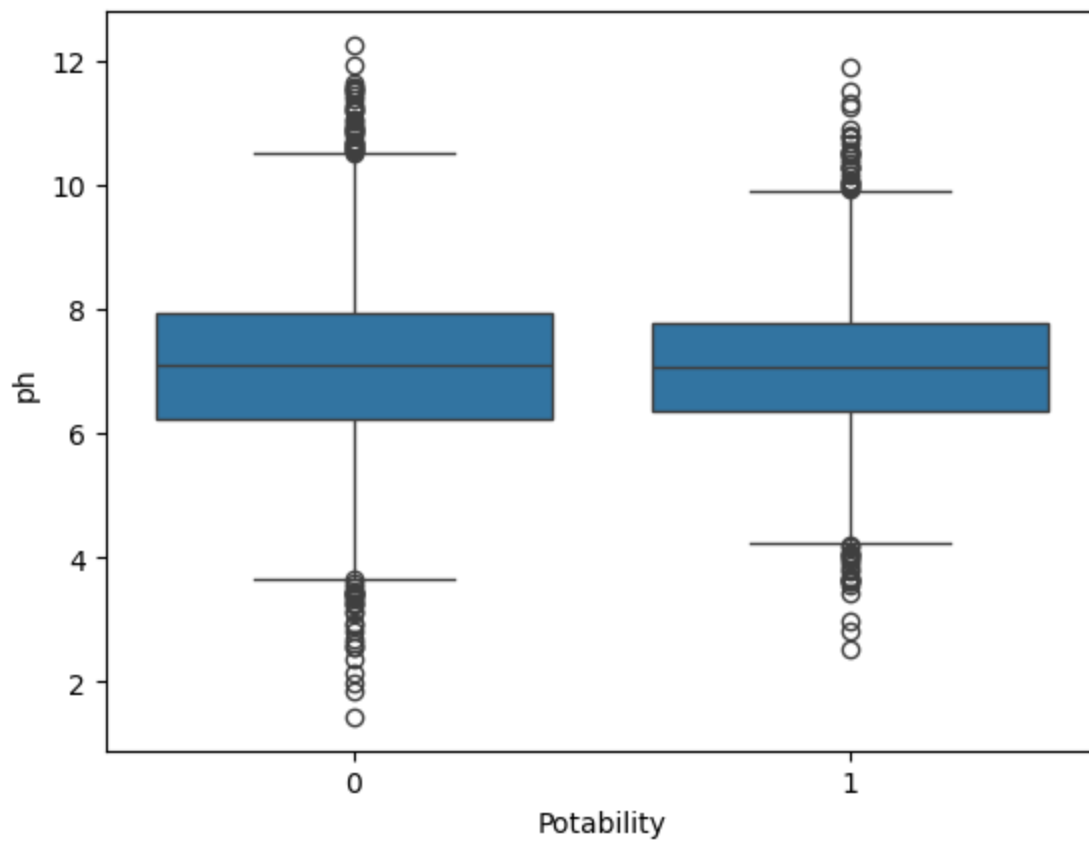
بعد از توضیح Boxplot سراغ نمایش outliers و حذف آن‌ها می‌رویم.

حذف مقادیر پرت از ستون Ph

```
sns.boxplot(df,x='Potability',y='ph')
```

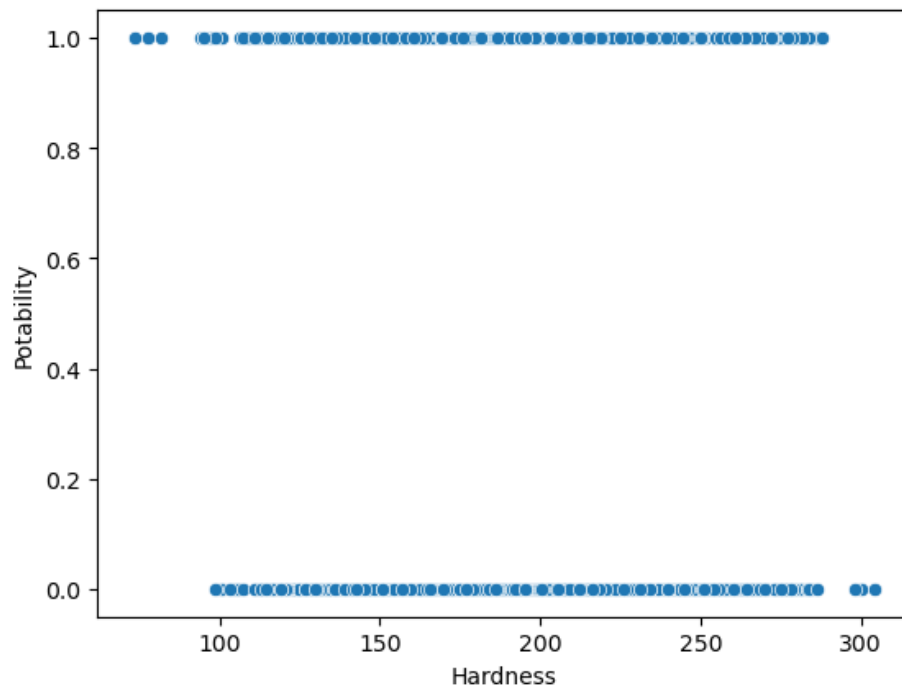


```
drop_index=df[(df['ph']>13) & (df['Potability']==0)].index
df=df.drop(drop_index,axis=0)
drop_index=df[(df['ph']>13) & (df['Potability']==1)].index
df=df.drop(drop_index,axis=0)
drop_index=df[(df['ph']<1) & (df['Potability']==0)].index
df=df.drop(drop_index,axis=0)
drop_index=df[(df['ph']<2) & (df['Potability']==1)].index
df=df.drop(drop_index,axis=0)
sns.boxplot(df,x='Potability',y='ph')
```



حذف مقادیر پرت از ستون Hardness

```
sns.scatterplot(df,x='Hardness',y='Potability')
```

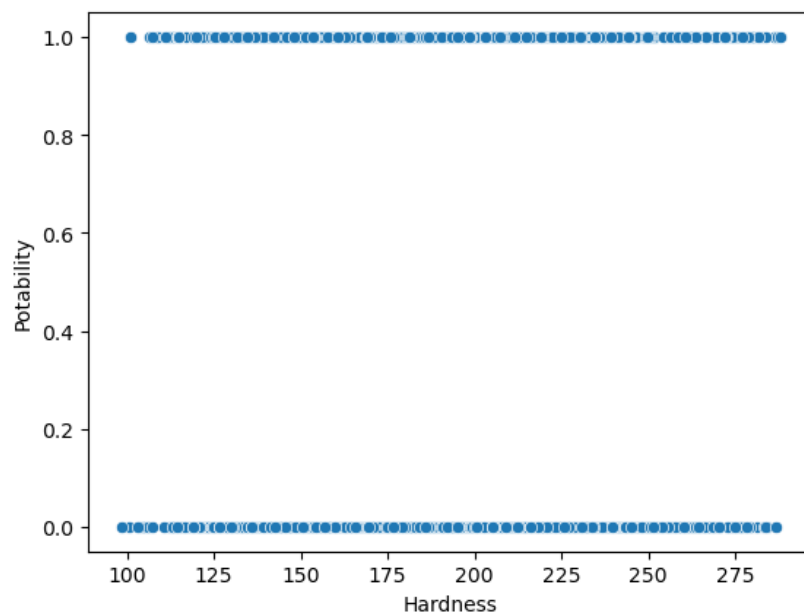


```

drop_index=df[(df['Hardness']>290) & (df['Potability']==1)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Hardness',y='Potability')
drop_index=df[(df['Hardness']<100) & (df['Potability']==1)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Hardness',y='Potability')
drop_index=df[(df['Hardness']>290) & (df['Potability']==0)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Hardness',y='Potability')

```

نتیجه:

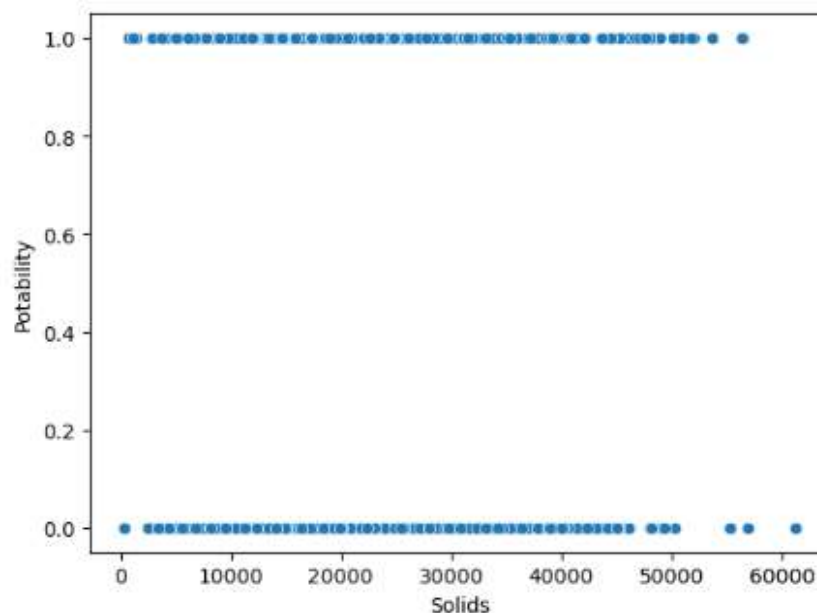


حذف مقادیر پرت از ستون Solids

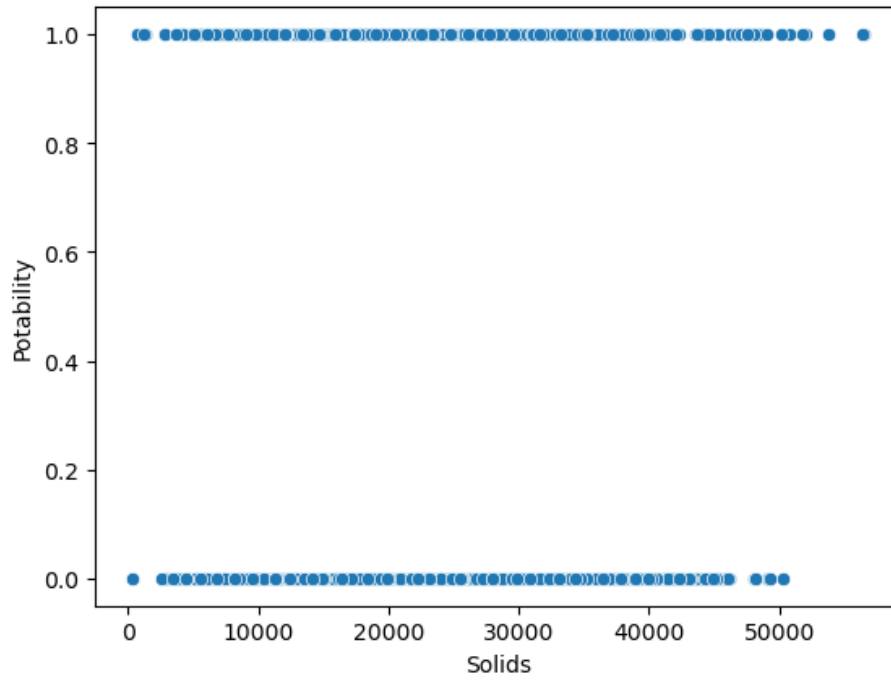
```

sns.scatterplot(df,x='Solids',y='Potability')

```

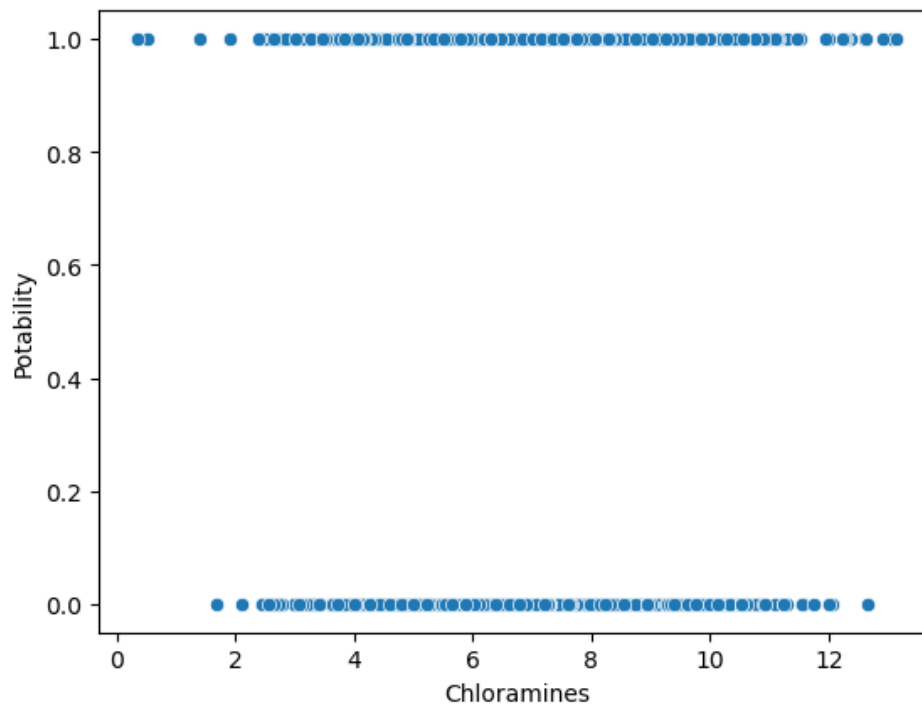


```
drop_index=df[(df['Solids']>55000) & (df['Potability']==0)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Solids',y='Potability')
```

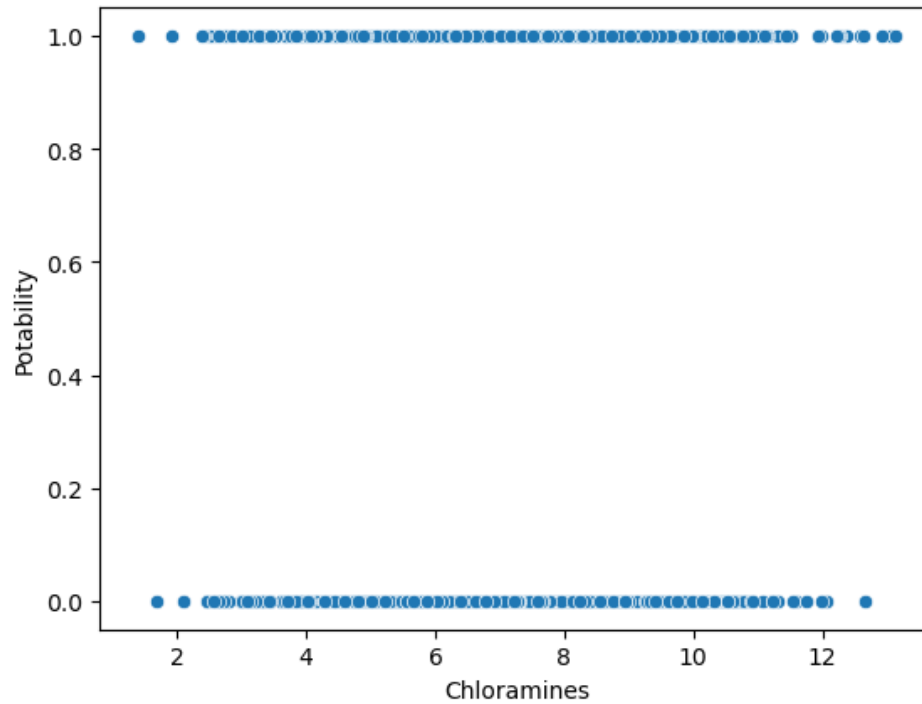


حذف مقادیر پرت از ستون Chloramines

```
sns.scatterplot(df,x='Chloramines',y='Potability')
```

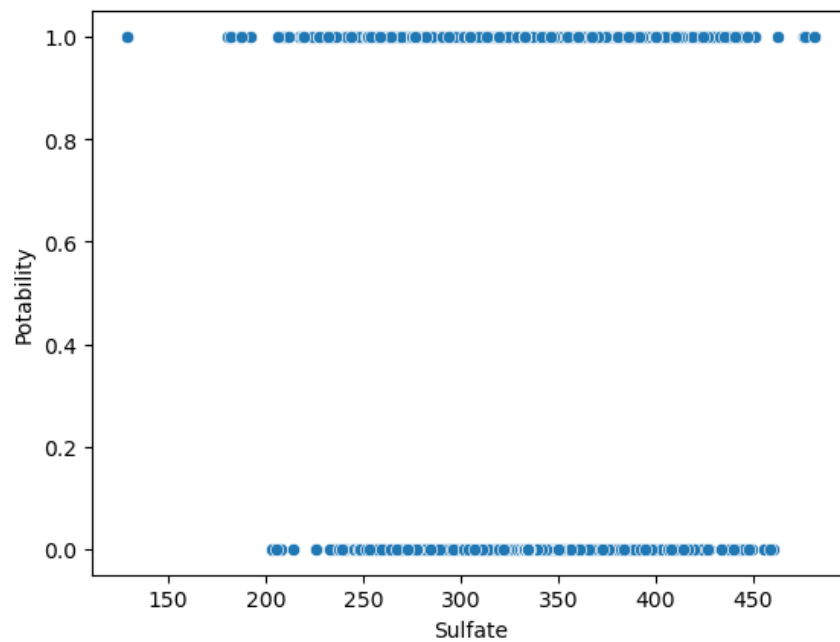


```
drop_index=df[(df['Chloramines']<0.6) & (df['Potability']==1)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Chloramines',y='Potability')
```

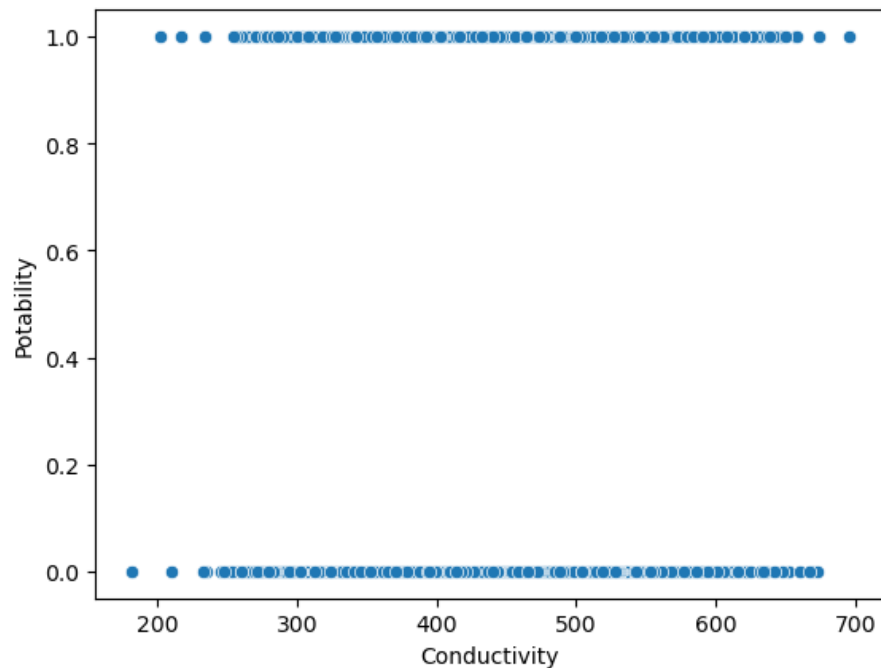


حذف مقادیر پرت از ستون Sulfate

```
sns.scatterplot(df,x='Sulfate',y='Potability')
```



```
drop_index=df[(df['Conductivity']>700) & (df['Potability']==0)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Conductivity',y='Potability')
```



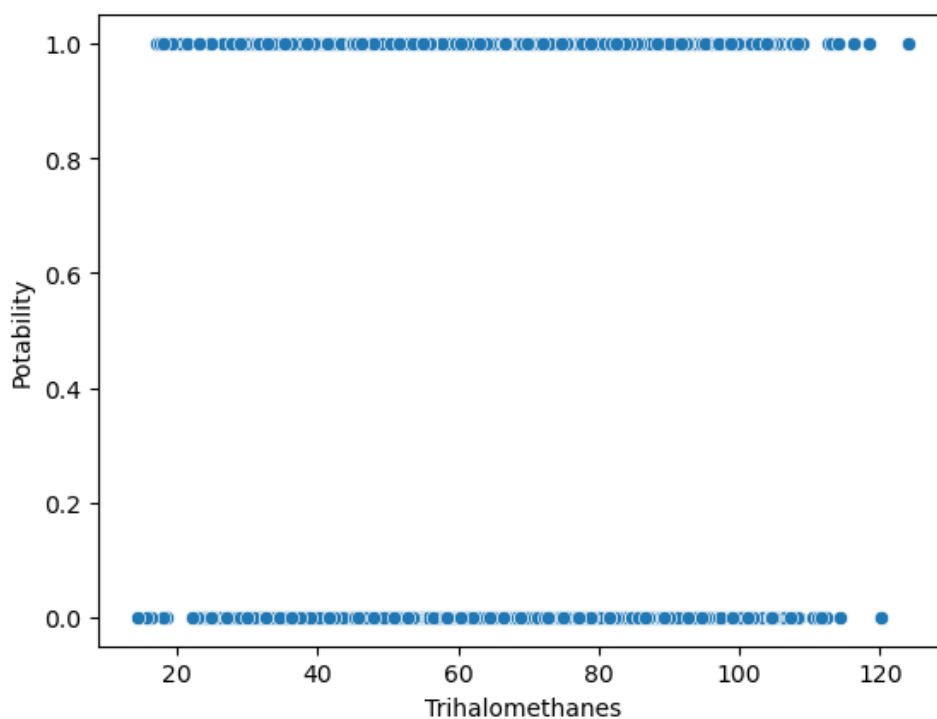
حذف مقادیر پرت از ستون Organic_carbon

```
drop_index=df[(df['Organic_carbon']<3) & (df['Potability']==1)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Organic_carbon',y='Potability')
drop_index=df[(df['Organic_carbon']>25) & (df['Potability']==0)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Organic_carbon',y='Potability')
```



حذف مقادیر پرت از ستون Trihalomethanes

```
drop_index=df[(df['Trihalomethanes']<1) & (df['Potability']==0)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Trihalomethanes',y='Potability')
drop_index=df[(df['Trihalomethanes']<10) & (df['Potability']==1)].index
df=df.drop(drop_index,axis=0)
sns.scatterplot(df,x='Trihalomethanes',y='Potability')
```



مزایای حذف داده‌های پرت:

- کاهش نویز داده‌ها
- افزایش دقت مدل
- بهبود قابلیت تعمیم مدل

تبدیل داده‌های Object به داده‌های عددی

در این گام بررسی میکنیم که کدام ستون‌ها object هستند.

```
df.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3276 entries, 0 to 3275
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ph                    3276 non-null   float64
1   Hardness              3276 non-null   float64
2   Solids                3276 non-null   float64
3   Chloramines           3276 non-null   float64
4   Sulfate               3276 non-null   float64
5   Conductivity          3276 non-null   float64
6   Organic_carbon        3276 non-null   float64
7   Trihalomethanes       3276 non-null   float64
8   Turbidity             3276 non-null   float64
9   Potability            3276 non-null   int64
dtypes: float64(9), int64(1)
memory usage: 256.1 KB

numeric_columns=df.select_dtypes(include=['int64','Float64']).columns
numeric_columns
✓ 0.0s

Index(['ph', 'Hardness', 'Solids', 'Chloramines', 'Sulfate', 'Conductivity',
      'Organic_carbon', 'Trihalomethanes', 'Turbidity', 'Potability'],
      dtype='object')
```

```
print(df.dtypes)
✓ 0.0s

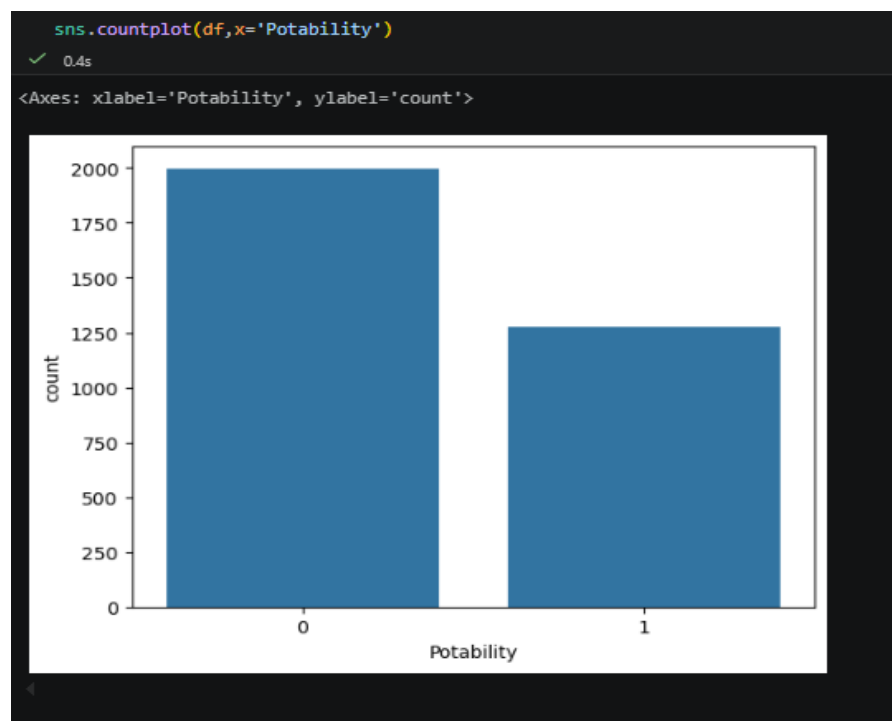
ph                float64
Hardness          float64
Solids            float64
Chloramines       float64
Sulfate           float64
Conductivity      float64
Organic_carbon    float64
Trihalomethanes   float64
Turbidity         float64
Potability        int64
dtype: object
```


نشان داده شد که تمامی ستون ها عددی هستند پس نیاز به استفاده از One و Label Encoding و Hot Encoding و Get_dummies نیست.

توصیف خروجی

مقادیر تابع هدف فقط صفر و یک هستند.

```
print(df['Potability'].value_counts())  
print(df['Potability'].value_counts(normalize=True)*100)|  
✓ 0.0s  
Potability  
0    1998  
1    1278  
Name: count, dtype: int64  
Potability  
0    60.989011  
1    39.010989  
Name: proportion, dtype: float64  
  
df['Potability'].unique()  
✓ 0.0s  
array([0, 1])
```



خوشه‌بندی داده‌ها (Clustering)

به منظور بررسی ساختار پنهان داده‌ها از الگوریتم K-Means استفاده شد. هدف از این بخش، تحلیل بدون ناظر داده‌ها و بررسی شباهت نمونه‌ها پیش از انجام طبقه‌بندی بود.

خوشه‌بندی یکی از روش‌های یادگیری بدون ناظر (Unsupervised Learning) است که هدف آن گروه‌بندی داده‌های مشابه در یک خوشه و جداسازی داده‌های غیرمشابه در خوشه‌های مختلف می‌باشد. در این روش برخلاف مسائل طبقه‌بندی، برچسب خروجی مشخصی وجود ندارد و الگوریتم تنها بر اساس شباهت میان نمونه‌ها عمل می‌کند. در این پروژه از الگوریتم K-Means برای خوشه‌بندی داده‌ها استفاده شد. این الگوریتم داده‌ها را به K خوشه تقسیم می‌کند به گونه‌ای که مجموع فاصله نقاط هر خوشه تا مرکز آن خوشه حداقل شود.

مراحل اجرای الگوریتم K-Means به صورت زیر است:

۱. انتخاب تعداد خوشه‌ها (K)
۲. تعیین تصادفی مراکز اولیه خوشه‌ها
۳. تخصیص هر نمونه به نزدیک‌ترین مرکز خوشه
۴. محاسبه مجدد مرکز هر خوشه
۵. تکرار مراحل ۳ و ۴ تا زمان همگرایی

تابع هدف در الگوریتم K-Means به صورت زیر تعریف می‌شود:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

که در آن:

- (K) تعداد خوشه‌ها است.
- (C_i) مجموعه نقاط متعلق به خوشه (i) است.
- (μ_i) مرکز خوشه (i) است.
- $(\|x - \mu_i\|^2)$ فاصله اقلیدسی بین نقطه و مرکز خوشه را نشان می‌دهد.

هدف الگوریتم کمینه کردن مقدار تابع (J) است؛ یعنی نقاط هر خوشه تا حد امکان به مرکز خوشه خود نزدیک باشند.

برای تعیین تعداد مناسب خوشه‌ها از روش Elbow استفاده شد. در این روش مقدار Inertia یا مجموع مربعات فاصله نقاط از مرکز خوشه برای مقادیر مختلف K محاسبه می‌شود. سپس نمودار Inertia بر حسب K رسم می‌گردد. نقطه‌ای که کاهش خطا پس از آن کند می‌شود، به عنوان تعداد مناسب خوشه‌ها انتخاب می‌شود. استفاده از خوشه‌بندی در این پروژه با هدف بررسی ساختار پنهان داده‌ها، شناسایی گروه‌های مشابه نمونه‌های آب و تحلیل توزیع طبیعی داده‌ها انجام شد. نتایج خوشه‌بندی نشان می‌دهد که داده‌ها دارای الگوهای مشخصی هستند و برخی نمونه‌ها از نظر ویژگی‌های فیزیکی و شیمیایی شباهت زیادی به یکدیگر دارند.

کد

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_cluster = df.drop("Potability", axis=1)

sc_cluster = StandardScaler()

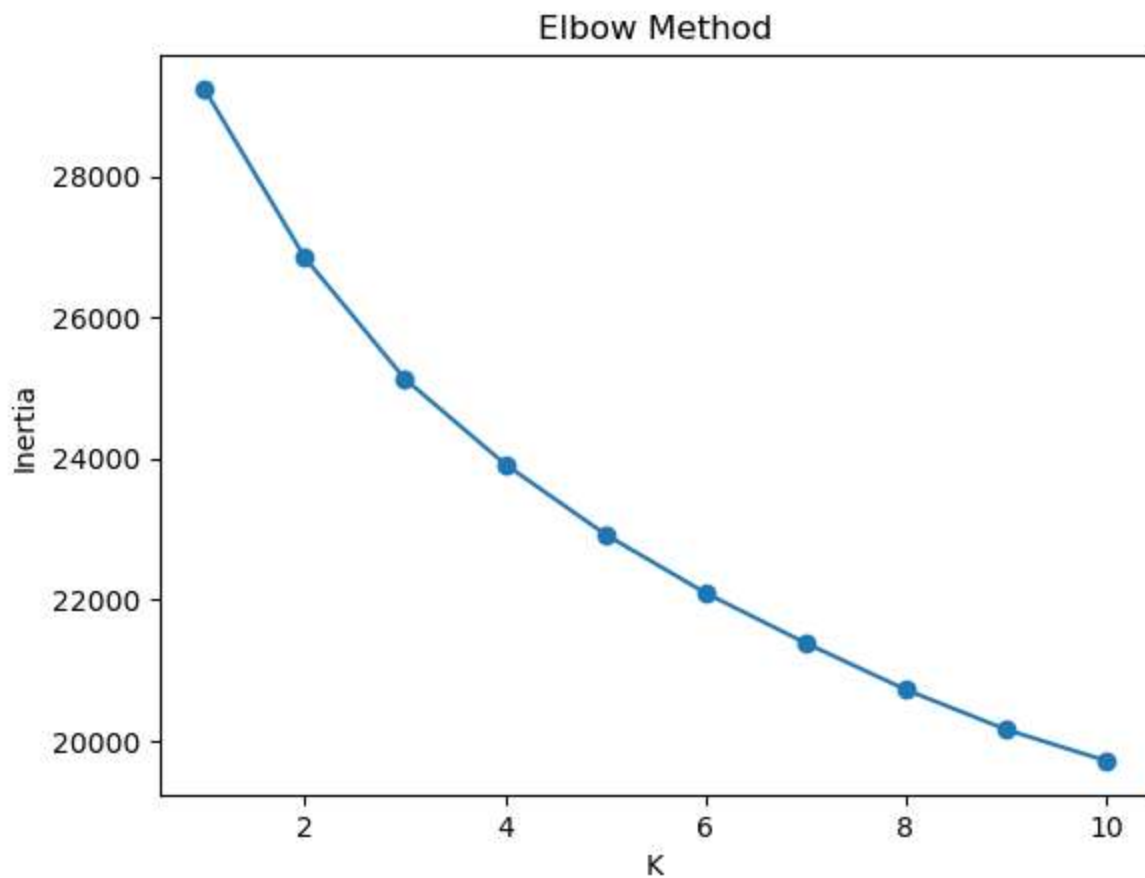
X_cluster_scaled = sc_cluster.fit_transform(X_cluster)

inertia = []

for k in range(1,11):

    km = KMeans(
        n_clusters=k,
```

```
        random_state=42,  
        n_init=10  
    )  
  
    km.fit(X_cluster_scaled)  
  
    inertia.append(km.inertia_)  
  
plt.plot(  
    range(1,11),  
    inertia,  
    marker='o'  
)  
  
plt.xlabel("K")  
plt.ylabel("Inertia")  
plt.title("Elbow Method")  
plt.show()
```



تحلیل همبستگی ویژگی‌ها

برای بررسی ارتباط بین متغیرها از ماتریس همبستگی استفاده شد.

```
plt.figure(figsize=(10,6))

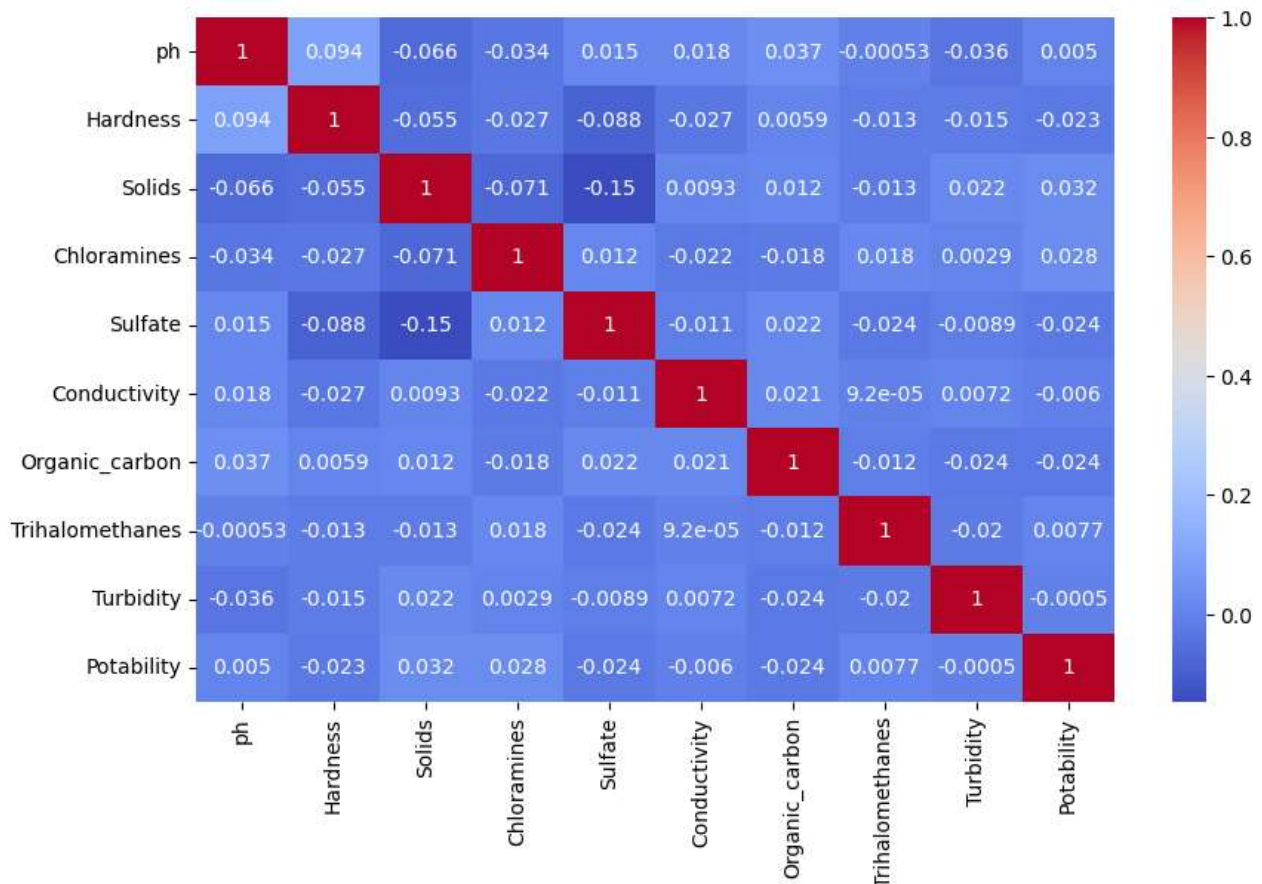
sns.heatmap(
    df.corr(),
    annot=True,
    cmap='coolwarm'
)

plt.show()
```

✓ 0.8s

در این بخش مشخص شد که برخی ویژگی‌ها دارای همبستگی ضعیف و برخی دارای همبستگی متوسط با متغیر هدف هستند.

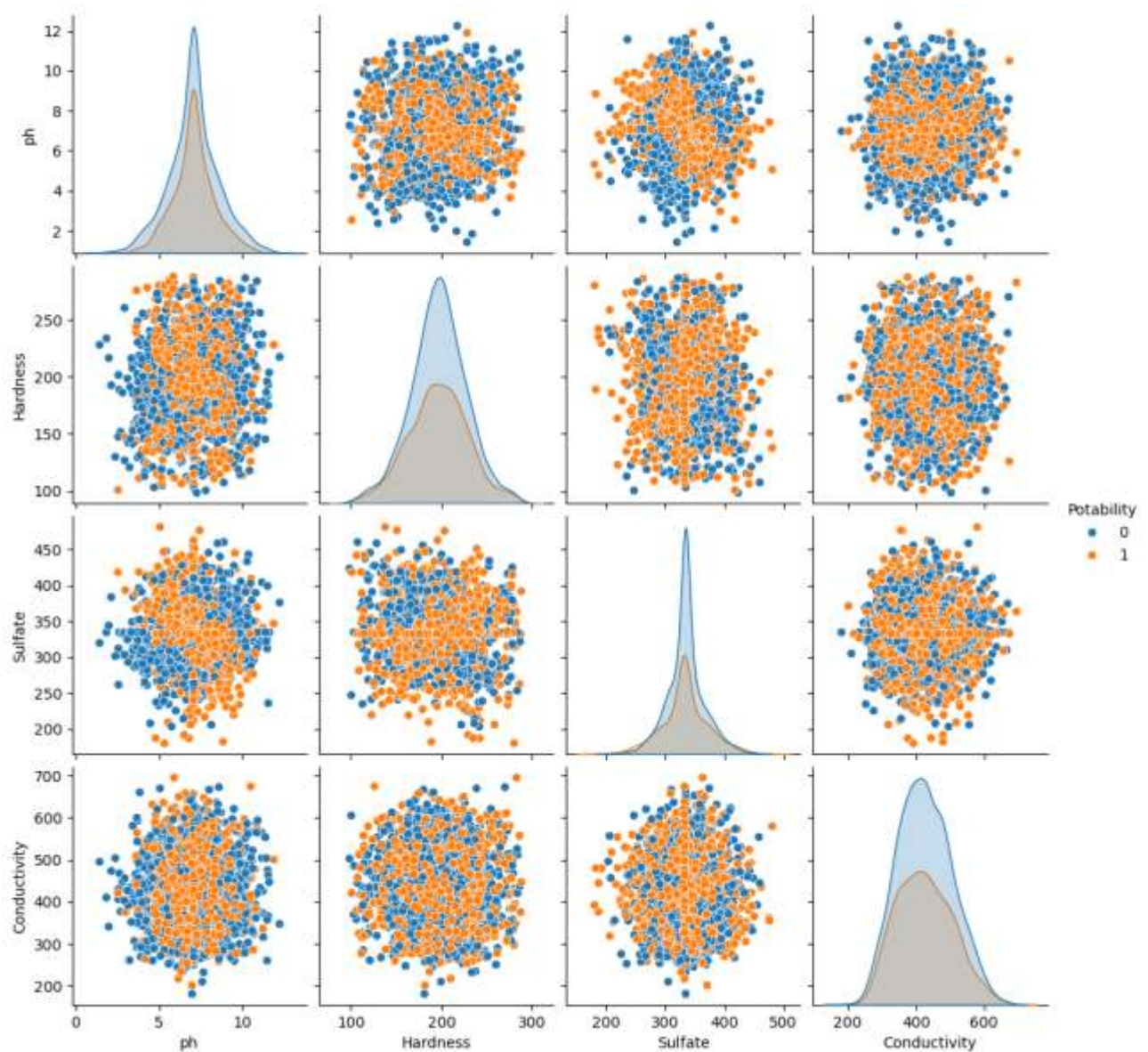
نمایش همبستگی با Heatmap



```
sns.pairplot(
    df[
        ['ph',
         'Hardness',
         'Sulfate',
         'Conductivity',
         'Potability']
    ],
    hue='Potability'
)

plt.show()
```

نمایش همبستگی با Pairplot



تعیین ورودی و خروجی

- X شامل ویژگی‌های ورودی است.
- Y همان خروجی که شامل آشامیدنی یا غیرآشامیدنی بودن آب است. چون خروجی صفر و یکی است در نتیجه اط classification و متدهای آن استفاده می‌کنیم.

```
X = df.drop('Potability',axis=1)
y = df['Potability']
```

سپس داده‌ها به دو بخش آموزش و آزمون تقسیم شدند:

```
from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.1,
    random_state=42,
    stratify=y
)
```

در این پروژه 80 درصد داده‌ها برای آموزش و 20 درصد برای تست استفاده شد.

استانداردسازی (Standardization) و نرمال‌سازی (Normalization)

در مسائل یادگیری ماشین، ویژگی‌های مختلف معمولاً دارای مقیاس‌های متفاوتی هستند. برای مثال ممکن است یک ویژگی در بازه ۰ تا ۱۴ و ویژگی دیگر در بازه ۰ تا ۵۰۰۰۰ قرار داشته باشد. این اختلاف مقیاس می‌تواند باعث شود برخی الگوریتم‌ها اهمیت بیشتری به ویژگی‌های با مقادیر بزرگ‌تر بدهند. به همین دلیل قبل از آموزش مدل، داده‌ها مقیاس‌بندی می‌شوند.

• نرمال سازی (Normalization)

در روش نرمال سازی، مقادیر هر ویژگی به بازه مشخصی (معمولاً بین ۰ و ۱) منتقل می شوند.

فرمول نرمال سازی Min-Max به صورت زیر است:

$$x_{new} = \frac{(x - x_{min})}{(x_{max} - x_{min})}$$

که در آن:

- مقدار اولیه داده x
- کمترین مقدار ویژگی x_{min}
- بیشترین مقدار ویژگی x_{max}
- مقدار نرمال شده x_{new}

مزیت این روش حفظ شکل توزیع داده ها است، اما نسبت به داده های پرت (Outlier) حساس می باشد.

• استاندارد سازی (Standardization)

در استاندارد سازی، داده ها به گونه ای تبدیل می شوند که میانگین آن ها برابر صفر و انحراف معیار آن ها برابر یک شود. در این پروژه از روش StandardScaler استفاده شد.

فرمول استاندارد سازی:

$$x = \frac{(X - \mu)}{\sigma}$$

که در آن:

- مقدار اولیه ویژگی X
- میانگین ویژگی μ
- انحراف معیار ویژگی σ
- مقدار استاندارد شده x

پس از اعمال این تبدیل:

Mean ≈ 0
Standard Deviation ≈ 1

خواهد شد.

کد

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

ابتدا میانگین و انحراف معیار روی داده‌های آموزشی محاسبه می‌شود (fit) و سپس همان مقادیر برای تبدیل داده‌های آموزشی و آزمون مورد استفاده قرار می‌گیرند (transform). این کار از نشت اطلاعات (Data Leakage) جلوگیری می‌کند.

دلیل استفاده از StandardScaler در این پروژه

الگوریتم‌های زیر به مقیاس داده‌ها حساس هستند:

- KNN
- SVM (SVC)

• Logistic Regression

بنابراین قبل از آموزش این مدل‌ها از StandardScaler استفاده شد تا تمامی ویژگی‌ها در یک مقیاس یکسان قرار گیرند. این کار باعث افزایش سرعت آموزش و بهبود عملکرد مدل‌های طبقه‌بندی شد. استانداردسازی باعث می‌شود تمامی ویژگی‌ها دارای میانگین صفر و انحراف معیار یک باشند. این مرحله برای الگوریتم‌هایی مانند KNN و SVC اهمیت زیادی دارد.

ویژگی	Normalization	Standardization
بازه خروجی	معمولاً ۰ تا ۱	بدون بازه مشخص
فرمول	$x_{new} = \frac{(x - x_{min})}{(x_{max} - x_{min})}$	$x = \frac{(X - \mu)}{\sigma}$
حساسیت به Outlier	زیاد	کمتر
ابزار رایج در sklearn	MinMaxScaler	StandardScaler
کاربرد	شبکه‌های عصبی	KNN, SVM, Logistic Regression

مدل‌های طبقه‌بندی

در این پروژه پنج الگوریتم مختلف مورد استفاده قرار گرفتند:

Logistic Regression

رگرسیون لجستیک یکی از الگوریتم‌های یادگیری نظارت‌شده برای مسائل طبقه‌بندی است. این الگوریتم احتمال تعلق یک نمونه به یک کلاس خاص را محاسبه کرده و سپس بر اساس یک آستانه (معمولاً ۰.۵) کلاس نهایی را تعیین می‌کند. برخلاف رگرسیون خطی، خروجی این مدل بین صفر و یک قرار دارد و از تابع سیگموئید برای تبدیل خروجی خطی به احتمال استفاده می‌شود.

فرمول تابع سیگموئید:

$$P(y = 1|x) = \frac{1}{(1 + e^{-z})}$$

$$z = \beta_0 + \beta_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

مزایا:

- سادگی و سرعت بالا
- مناسب برای مسائل طبقه‌بندی دودویی
- قابلیت تفسیر ضرایب مدل

معایب:

- فرض وجود رابطه تقریباً خطی بین ویژگی‌ها و خروجی
- عملکرد ضعیف‌تر در روابط بسیار پیچیده و غیر خطی

کد

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['lbfgs', 'liblinear'],
    'class_weight': [None, 'balanced']
}

grid = GridSearchCV(
    LogisticRegression(max_iter=5000),
    param_grid,
    cv=10,
    scoring='accuracy',
    n_jobs=-1
)
```

```

grid.fit(X_train,y_train)
y_pred = grid.predict(X_test)

print(grid.best_params_)
from sklearn.metrics import
accuracy_score,confusion_matrix,classification_report,ConfusionMatrixDisplay
print(classification_report(y_test,y_pred))

```

{'C': 0.01, 'class_weight': None, 'solver': 'liblinear'}

	precision	recall	f1-score	support
0	0.61	1.00	0.76	199
1	0.00	0.00	0.00	126
accuracy			0.61	325
macro avg	0.31	0.50	0.38	325
weighted avg	0.37	0.61	0.47	325

K-Nearest Neighbors (KNN)

در این الگوریتم، کلاس هر نمونه بر اساس نزدیک‌ترین همسایه‌های آن تعیین می‌شود. الگوریتم KNN یکی از ساده‌ترین روش‌های طبقه‌بندی است که بر اساس شباهت نمونه‌ها عمل می‌کند. در این روش، برای هر نمونه جدید، نزدیک‌ترین K همسایه در داده‌های آموزشی پیدا شده و کلاس غالب میان آن‌ها به عنوان خروجی انتخاب می‌شود. معیار اصلی در KNN فاصله بین نمونه‌ها است که معمولاً از فاصله اقلیدسی استفاده می‌شود.

$$d(x,y) = \sqrt{[(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2]}$$

سپس کلاس نمونه از طریق رأی‌گیری K همسایه نزدیک تعیین می‌شود:

$$\hat{y} = \text{Majority Vote}(K \text{ Nearest Neighbors})$$

مزایا:

- ساده و قابل فهم
- عدم نیاز به مرحله آموزش پیچیده
- مناسب برای داده‌های غیرخطی

معایب:

- حساسیت به مقیاس ویژگی‌ها
- هزینه محاسباتی بالا در داده‌های بزرگ
- حساسیت به انتخاب مقدار K

به همین دلیل در این پروژه قبل از استفاده از KNN از StandardScaler استفاده شد.

کد

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

knn = KNeighborsClassifier()

param_grid = {
    'n_neighbors': [3,5,7,9,11,13,15,17,19],
    'weights': ['uniform','distance'],
    'metric': ['euclidean','manhattan','minkowski']
}

grid = GridSearchCV(
    knn,
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid.fit(X_train,y_train)

print("Best Parameters:")
print(grid.best_params_)
```

```

print("\nBest Accuracy:")
print(grid.best_score_)

best_knn = grid.best_estimator_

y_pred = best_knn.predict(X_test)

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

print("Accuracy =",accuracy_score(y_test,y_pred))

print(classification_report(y_test,y_pred))

from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(
    best_knn,
    X_test,
    y_test
)

```

Best Parameters:

```
{'metric': 'manhattan', 'n_neighbors': 19, 'weights': 'uniform'}
```

Best Accuracy:

```
0.6604021777309448
```

```
Accuracy = 0.6369230769230769
```

```
precision recall f1-score support
```

```

0    0.65    0.90    0.75    199
1    0.59    0.21    0.31    126

```

```

accuracy                0.64    325
macro avg              0.62    0.56    0.53    325
weighted avg           0.62    0.64    0.58    325

```

Support Vector Machine (SVC)

این مدل با یافتن بهترین مرز تصمیم میان کلاس‌ها عمل طبقه‌بندی را انجام می‌دهد. ماشین بردار پشتیبان یکی از قدرتمندترین الگوریتم‌های طبقه‌بندی است. هدف این الگوریتم یافتن یک ابرصفحه (Hyperplane) بهینه است که بیشترین فاصله را از نمونه‌های دو کلاس داشته باشد.

فاصله بین نزدیک‌ترین نمونه‌ها و ابرصفحه را Margin می‌نامند. SVM سعی می‌کند این فاصله را بیشینه کند.

فرمول ابرصفحه تصمیم:

$$w \cdot x + b = 0$$

که در آن:

- w بردار وزن‌ها
- x بردار ویژگی‌ها
- b مقدار بایاس

هدف SVC بیشینه کردن margin است.

$$\text{Margin} = \frac{2}{\|w\|}$$

در این پروژه از کرنل RBF نیز استفاده شد که قادر است روابط غیرخطی را مدل‌سازی کند.

فرمول کرنل RBF :

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

مزایا:

- دقت بالا در بسیاری از مسائل طبقه‌بندی
- توانایی مدل‌سازی روابط غیرخطی
- مقاومت مناسب در برابر بیش‌برازش

معایب:

- زمان آموزش نسبتاً زیاد
- حساسیت به انتخاب پارامترهای C و γ

کد

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

svc = SVC()

param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear']
}

grid = GridSearchCV(
    svc,
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best Parameters:")
print(grid.best_params_)

print("\nBest Accuracy:")
print(grid.best_score_)
best_svc = grid.best_estimator_
```



```

y_pred = best_svc.predict(X_test)
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

print("Accuracy =", accuracy_score(y_test, y_pred))

print(classification_report(y_test, y_pred))
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(
    best_svc,
    X_test,
    y_test
)

```

Best Parameters:

{'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}

Accuracy = 0.65527950310559

	precision	recall	f1-score	support
0	0.65	0.93	0.77	198
1	0.67	0.21	0.32	124
accuracy		0.66		322
macro avg	0.66	0.57	0.54	322
weighted avg	0.66	0.66	0.60	322

Random Forest

الگوریتمی مبتنی بر مجموعه‌ای از درختان تصمیم که معمولاً عملکرد بسیار مناسبی در مسائل طبقه بندی دارد. جنگل تصادفی یک الگوریتم مبتنی بر مجموعه‌ای از درخت‌های تصمیم است. در این روش چندین درخت تصمیم به صورت مستقل آموزش داده می‌شوند و نتیجه نهایی از طریق رأی گیری اکثریت تعیین می‌شود.

هر درخت روی بخشی تصادفی از داده‌ها و ویژگی‌ها آموزش می‌بیند که باعث افزایش تنوع مدل‌ها و کاهش بیش‌برازش می‌شود.

فرمول رأی‌گیری نهایی:

$$\hat{y} = \text{mode}(T_1(x). T_2(x) \dots T_n(x))$$

که در آن:

- T_1 تا T_n درخت‌های تصمیم
- mode کلاس دارای بیشترین رأی

معیار تقسیم درخت معمولاً بر اساس Gini Index است:

$$Gini = 1 - \sum (p_i^2)$$

$p_i = \text{probabilities of class } i$

مزایا:

- دقت بالا
- مقاومت مناسب در برابر نویز
- کاهش خطر Overfitting نسبت به یک درخت تصمیم منفرد

معایب:

- پیچیدگی بیشتر نسبت به درخت تصمیم
- دشواری تفسیر مدل

کد

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(
    class_weight='balanced', random_state=42
)

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid = GridSearchCV(
    rf,
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best Parameters:")
print(grid.best_params_)

print("\nBest Accuracy:")
print(grid.best_score_)

best_rf = grid.best_estimator_

y_pred = best_rf.predict(X_test)

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

print("Accuracy =", accuracy_score(y_test, y_pred))

print(classification_report(y_test, y_pred))

from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(
    best_rf,
    X_test,
```

```

y_test
)
Best Parameters:
{'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2,
'n_estimators': 100}
Accuracy = 0.8167701863354038
      precision  recall  f1-score  support
0      0.82    0.90    0.86    198
1      0.82    0.68    0.74    124
accuracy                0.82    322
macro avg    0.82    0.79    0.80    322
weighted avg    0.82    0.82    0.81    322

```

AdaBoost

یک روش Ensemble که چندین مدل ضعیف را ترکیب کرده و یک مدل قدرتمند ایجاد می‌کند.

AdaBoost یکی از الگوریتم‌های Ensemble Learning است که چندین مدل ضعیف (Weak Learner) را به یک مدل قوی تبدیل می‌کند. معمولاً از درخت‌های تصمیم کوچک (Decision Stump) به عنوان یادگیرنده پایه استفاده می‌شود. ایده اصلی AdaBoost این است که در هر مرحله وزن نمونه‌هایی که اشتباه طبقه‌بندی شده‌اند افزایش یابد تا مدل بعدی تمرکز بیشتری روی آن‌ها داشته باشد. وزن هر مدل بر اساس میزان خطای آن محاسبه می‌شود.

محاسبه‌ی خطای مدل:

$$\text{error} = \sum w_i I(y_i \neq h_i(x_i))$$

فرمول وزن مدل:

$$\alpha = \frac{1}{2} \times \ln\left(\frac{1 - \text{error}}{\text{error}}\right)$$

که در آن:

- error میزان خطای مدل
- α وزن اختصاص داده شده به مدل

پیش‌بینی نهایی به صورت مجموع وزن‌دار خروجی تمام مدل‌ها انجام می‌شود.

$$F(x) = \sum \alpha_m h_m(x)$$

که در آن:

- $h_m(x)$ خروجی یادگیرنده m ام
- α_m وزن یادگیرنده m ام

که در آن:

- $h_m(x)$ خروجی مدل m
- α_m وزن مدل m

مزایا:

- بهبود دقت مدل‌های ضعیف
- عملکرد مناسب در بسیاری از مسائل طبقه‌بندی

معایب:

- حساسیت زیاد به داده‌های پرت و نویز
- امکان افت عملکرد در داده‌های نامتوازن

در این پروژه پنج الگوریتم مختلف شامل Logistic Regression ، KNN ، SVC ، Random Forest و AdaBoost مورد بررسی قرار گرفتند. هدف از استفاده همزمان از چند مدل، مقایسه عملکرد آنها و انتخاب بهترین الگوریتم برای پیش‌بینی قابلیت آشامیدن آب بود. همچنین برای بهینه‌سازی پارامترهای هر مدل از روش GridSearchCV و اعتبارسنجی متقابل پنج‌بخشی (5-Fold Cross Validation) استفاده شد تا بهترین تنظیمات ممکن برای هر الگوریتم تعیین گردد.

کد

```
param_ada = {
    'n_estimators':[50,100,200,300],
    'learning_rate':[0.01,0.1,0.5,1]
}

grid_ada = GridSearchCV(
    AdaBoostClassifier(
        random_state=42
    ),
    param_ada,
    cv=5,
    scoring='f1',
    n_jobs=-1
)

grid_ada.fit(
    X_train,
    y_train
)

best_ada = grid_ada.best_estimator_
models = {

    "AdaBoost":best_ada
}

for name,model in models.items():

    pred = model.predict(X_test)

    print("="*60)
    print(name)
```

```

print("Accuracy :",
      accuracy_score(y_test,pred))

print(
    classification_report(
        y_test,
        pred
    )
)

print(
    confusion_matrix(
        y_test,
        pred
    )
)

```

AdaBoost

Accuracy : 0.7267080745341615

	precision	recall	f1-score	support
0	0.73	0.87	0.80	198
1	0.71	0.49	0.58	124
accuracy			0.73	322
macro avg	0.72	0.68	0.69	322
weighted avg	0.72	0.73	0.71	322

[[173 25]

[63 61]]

Gradient Boosting

Gradient Boosting یکی از الگوریتم‌های Ensemble Learning است که با ترکیب چندین یادگیرنده ضعیف (Weak Learner)، یک مدل قدرتمند ایجاد می‌کند. در این روش، مدل‌ها به صورت ترتیبی آموزش داده می‌شوند؛ به این معنا که هر مدل جدید تلاش می‌کند خطاهای مدل قبلی را اصلاح کند. معمولاً درخت‌های تصمیم کم‌عمق به عنوان یادگیرنده پایه مورد استفاده قرار می‌گیرند.

برخلاف Random Forest که درخت‌ها را به صورت مستقل آموزش می‌دهد، در Gradient Boosting هر درخت جدید بر اساس خطاهای باقی‌مانده مدل‌های قبلی ساخته می‌شود. به همین دلیل این الگوریتم توانایی بسیار بالایی در یادگیری روابط پیچیده و غیرخطی بین ویژگی‌ها و خروجی دارد. در ابتدای فرآیند، یک مدل اولیه ساخته می‌شود که کمترین مقدار تابع خطا را تولید کند:

$$F_0(x) = \arg \min \sum L(y_i, \gamma)$$

در این رابطه:

- $F_0(x)$ مدل اولیه
- L تابع خطا (Loss Function)
- y_i مقدار واقعی خروجی
- γ مقدار ثابتی است که خطا را کمینه می‌کند.

سپس در هر مرحله یک مدل جدید به مدل قبلی اضافه می‌شود:

$$F_m(x) = F_{m-1}(x) + \alpha h_m(x)$$

که در آن:

- $F_m(x)$ مدل نهایی در مرحله m
- $F_{m-1}(x)$ مدل مرحله قبل
- $h_m(x)$ درخت تصمیم جدید
- α نرخ یادگیری (Learning Rate)

این فرمول نشان می‌دهد که مدل نهایی از جمع مدل قبلی و یک اصلاح‌کننده جدید تشکیل شده است. در واقع هر درخت جدید تلاش می‌کند بخشی از خطای باقی‌مانده را جبران کند.

نقش Learning Rate

پارامتر Learning Rate میزان تأثیر هر درخت جدید بر مدل نهایی را مشخص می‌کند.

- اگر مقدار آن کوچک باشد، یادگیری آهسته‌تر ولی پایدارتر خواهد بود.
- اگر مقدار آن بزرگ باشد، مدل سریع‌تر آموزش می‌بیند اما احتمال بیش‌برازش افزایش می‌یابد.

در فرمول زیر تأثیر Learning Rate مشخص است:

$$F_m(x) = F_{m-1}(x) + \alpha h_m(x)$$

هرچه مقدار α بزرگ‌تر باشد، سهم درخت جدید در پیش‌بینی نهایی بیشتر خواهد بود.

پارامترهای مهم Gradient Boosting

n_estimators

تعداد درخت‌های تصمیم مورد استفاده در فرآیند Boosting را مشخص می‌کند.

$$n_estimators = \text{Number of Trees}$$

افزایش تعداد درخت‌ها معمولاً باعث بهبود عملکرد مدل می‌شود اما زمان آموزش را نیز افزایش می‌دهد.

max_depth

حداکثر عمق هر درخت تصمیم را تعیین می‌کند.

max_depth = Depth of Tree

درخت‌های عمیق‌تر روابط پیچیده‌تری را یاد می‌گیرند اما احتمال Overfitting را افزایش می‌دهند.

subsample

درصد داده‌هایی را مشخص می‌کند که در هر مرحله برای آموزش درخت جدید استفاده می‌شوند.

subsample $\in (0,1]$

استفاده از مقادیر کمتر از یک باعث کاهش بیش‌برازش و افزایش قدرت تعمیم مدل می‌شود.

کد

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

param_gb = {
    'n_estimators': [50, 100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [2, 3, 4],
    'subsample': [0.8, 1.0]
}

grid_gb = GridSearchCV(
    GradientBoostingClassifier(
        random_state=42
    ),
    param_gb,
    cv=5,
```

```

        scoring='f1',
        n_jobs=-1
    )

    grid_gb.fit(
        X_train,
        y_train
    )

    best_gb = grid_gb.best_estimator_

    models = {
        "Gradient Boosting":best_gb
    }

    for name,model in models.items():

        pred = model.predict(X_test)

        print("="*60)
        print(name)

        print("Best Parameters:")
        print(grid_gb.best_params_)

        print("\nBest CV Score:")
        print(grid_gb.best_score_)

        print("\nAccuracy :",
              accuracy_score(y_test,pred))

        print(
            classification_report(
                y_test,
                pred
            )
        )

        print(
            confusion_matrix(
                y_test,
                pred
            )
        )

```

تنظیم پارامترها با GridSearchCV

برای یافتن بهترین پارامترهای هر مدل از GridSearchCV استفاده شد.

```
param_gb = {
    'n_estimators':[50,100,200,300],
    'learning_rate':[0.01,0.05,0.1,0.2],
    'max_depth':[2,3,4],
    'subsample':[0.8,1.0]
}

grid_gb = GridSearchCV(
    GradientBoostingClassifier(
        random_state=42
    ),
    param_gb,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
```

این روش تمامی ترکیب‌های ممکن پارامترها را بررسی کرده و بهترین ترکیب را بر اساس اعتبارسنجی متقابل انتخاب می‌کند.

```
=====
=
Gradient Boosting
Best Parameters:
{'learning_rate': 0.2, 'max_depth': 4, 'n_estimators': 200, 'subsample': 0.8}
Accuracy : 0.8043478260869565
      precision  recall  f1-score  support
0      0.82      0.87      0.85      198
1      0.77      0.70      0.73      124
accuracy                0.80      322
macro avg      0.80      0.79      0.79      322
weighted avg      0.80      0.80      0.80      322
[[172 26]
 [ 37 87]]
```

برای دیتاست Water Quality بهترین انتخاب این است که دو ویژگی نسبتاً مرتبط را انتخاب کنی، مثلاً:

- Sulfate
- Conductivity

یا

- Hardness
- Solids

یا

- pH
- Chloramines

من معمولاً Sulfate و Conductivity را پیشنهاد می‌کنم چون هر دو به کیفیت شیمیایی آب مربوط هستند.

بررسی رابطه بین دو ویژگی با استفاده از Clustering

به منظور بررسی رابطه بین دو ویژگی Sulfate و Conductivity از الگوریتم خوشه‌بندی K-Means استفاده شد. خوشه‌بندی یکی از روش‌های یادگیری بدون نظارت (Unsupervised Learning) است که داده‌های مشابه را در یک گروه قرار می‌دهد.

در این روش ابتدا تعداد خوشه‌ها برابر ۳ در نظر گرفته شد. سپس الگوریتم K-Means با محاسبه فاصله نمونه‌ها از مراکز خوشه، داده‌ها را به سه گروه تقسیم کرد. هدف این الگوریتم کمینه کردن مجموع فاصله نقاط هر خوشه تا مرکز آن خوشه است.

تابع هدف K-Means به صورت زیر تعریف می‌شود:

$$J = \sum_{i=1}^K \sum_{x \in C_i} ||x - \mu_i||^2$$

که در آن:

- K تعداد خوشه‌ها
- C_i خوشه i ام
- μ_i مرکز خوشه i ام
- $||x - \mu_i||^2$ مجذور فاصله اقلیدسی هر نمونه تا مرکز خوشه

نتایج نمودار خوشه‌بندی نشان داد که داده‌ها در چند گروه مجزا قرار می‌گیرند. این موضوع بیانگر وجود الگوهای متفاوت بین مقادیر Sulfate و Conductivity در نمونه‌های مختلف آب است. بنابراین می‌توان نتیجه گرفت که خوشه‌بندی قادر است ساختار پنهان داده‌ها و نحوه توزیع نمونه‌ها را مشخص کند و در تحلیل کیفیت آب مفید باشد.

▪ فاصله اقلیدسی (Euclidean Distance)

K-Means برای تخصیص هر نمونه به نزدیک‌ترین خوشه از فاصله اقلیدسی استفاده می‌کند:

$$d(x, y) = \sqrt{[(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2]}$$

که در آن:

- x و y دو نمونه از داده‌ها هستند .
- d فاصله بین دو نمونه را نشان می‌دهد .

هر نمونه به خوشه‌ای اختصاص داده می‌شود که کمترین فاصله را از مرکز آن داشته باشد.

▪ محاسبه مرکز خوشه (Centroid)

پس از تشکیل هر خوشه، مرکز آن از میانگین نقاط موجود در خوشه محاسبه می‌شود:

$$\mu_i = \left(\frac{1}{N_i} \right) \sum x_j$$

که در آن:

- μ_i مرکز خوشه i ام
- N_i تعداد نمونه‌های خوشه i ام
- x_j نمونه‌های موجود در خوشه

این مرکز در هر تکرار به‌روزرسانی می‌شود تا الگوریتم به همگرایی برسد.

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X_cluster = df[['Sulfate', 'Conductivity']]

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_cluster)

plt.figure(figsize=(8,6))

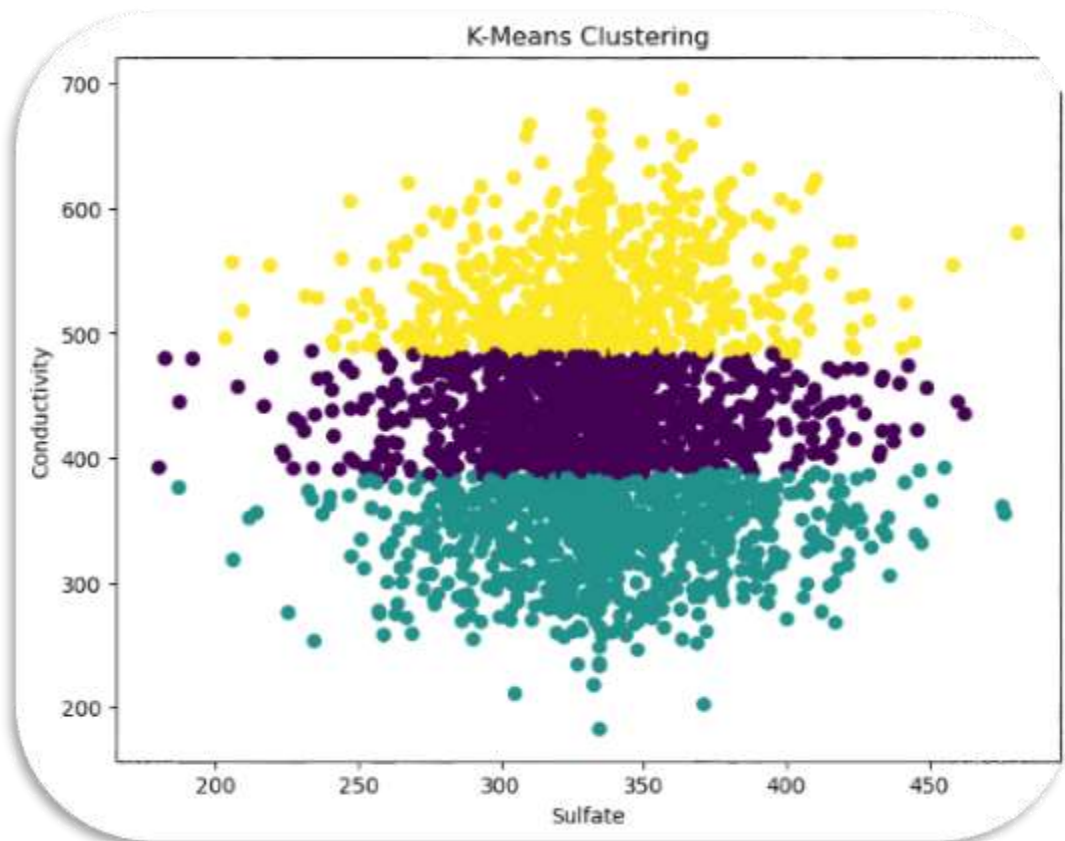
plt.scatter(
    X_cluster['Sulfate'],
    X_cluster['Conductivity'],
    c=clusters
)

plt.xlabel('Sulfate')
```

```
plt.ylabel('Conductivity')
plt.title('K-Means Clustering')

plt.show()
```

نتایج حاصل از کد



اگر بخواهی مرکز خوشه‌ها هم نمایش داده شوند:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

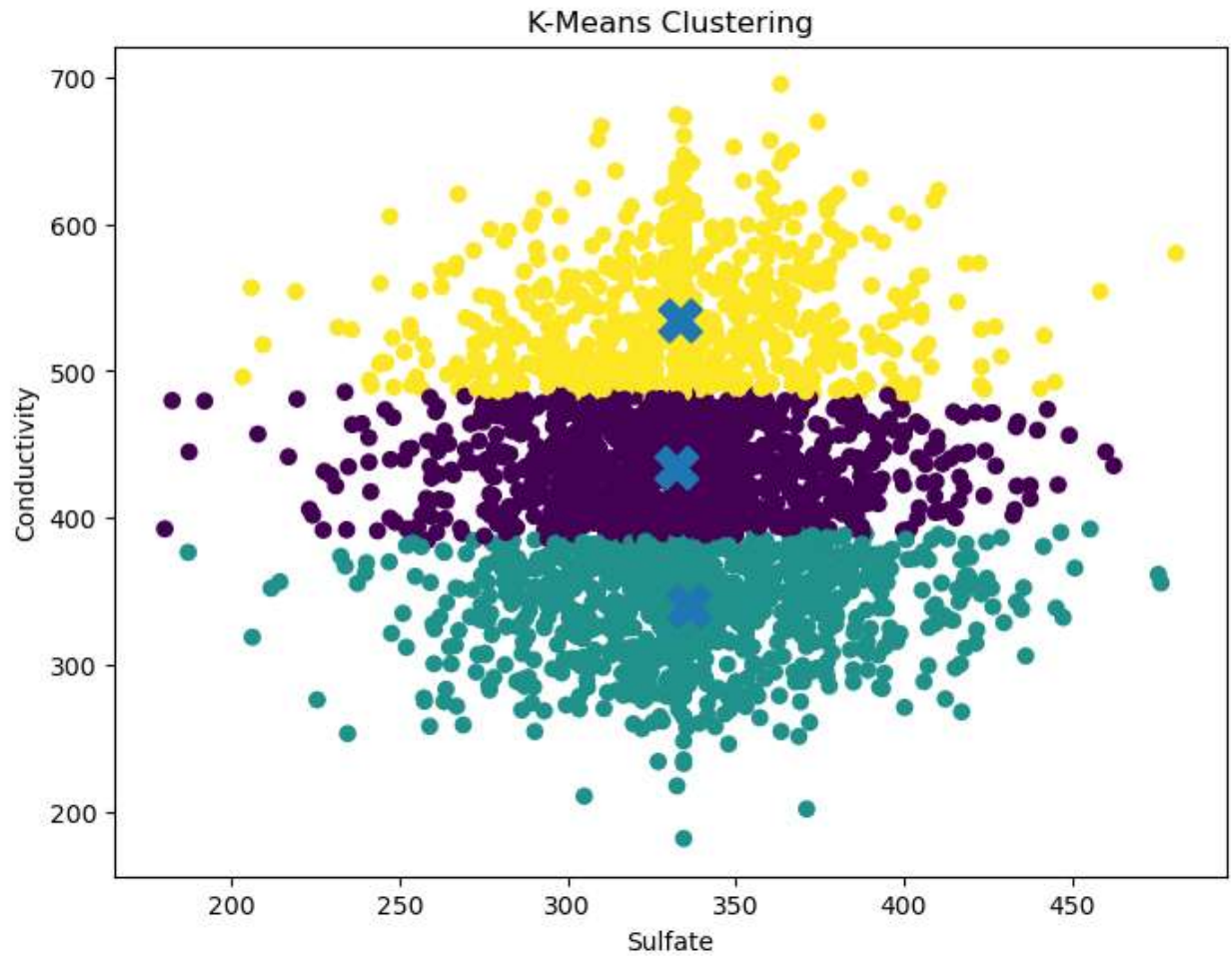
X_cluster = df[['Sulfate', 'Conductivity']]

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
```



```
)  
  
clusters = kmeans.fit_predict(X_cluster)  
  
centers = kmeans.cluster_centers_  
  
plt.figure(figsize=(8,6))  
  
plt.scatter(  
    X_cluster['Sulfate'],  
    X_cluster['Conductivity'],  
    c=clusters  
)  
  
plt.scatter(  
    centers[:,0],  
    centers[:,1],  
    marker='X',  
    s=300  
)  
  
plt.xlabel('Sulfate')  
plt.ylabel('Conductivity')  
plt.title('K-Means Clustering')  
  
plt.show()
```

نتایج حاصل از کد



ارزیابی مدل‌ها

برای ارزیابی عملکرد مدل‌ها از معیارهای زیر استفاده شد:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- Confusion Matrix Display
- Classification Report

Accuracy

دقت (Accuracy) نسبت تعداد پیش‌بینی‌های صحیح به کل نمونه‌ها را نشان می‌دهد. این معیار بیان می‌کند که مدل چه درصدی از داده‌ها را به درستی طبقه‌بندی کرده است.

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

Precision

معیار Precision نشان می‌دهد از میان نمونه‌هایی که مدل به عنوان کلاس مثبت پیش‌بینی کرده است، چه درصدی واقعاً مثبت بوده‌اند. این معیار زمانی اهمیت دارد که خطای (False Positive) مهم باشد.

$$Precision = \frac{TP}{(TP + FP)}$$

Recall

معیار Recall بیان می‌کند چه درصدی از نمونه‌های مثبت واقعی توسط مدل به درستی شناسایی شده‌اند. این معیار زمانی مهم است که از دست دادن نمونه‌های مثبت هزینه بالایی داشته باشد.

$$Recall = \frac{TP}{(TP + FN)}$$

F1-Score

معیار F1-Score میانگین هارمونیک Precision و Recall است و تعادلی بین این دو معیار برقرار می‌کند. این معیار به ویژه برای داده‌های نامتوازن مناسب است.

$$F1 = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)}$$

Confusion Matrix

ماتریس درهم‌ریختگی (Confusion Matrix) جدولی است که عملکرد مدل را با مقایسه مقادیر واقعی و پیش‌بینی شده نمایش می‌دهد. این ماتریس شامل چهار بخش اصلی است:

- **TP** پیش‌بینی مثبت صحیح
- **TN** پیش‌بینی منفی صحیح
- **FP** پیش‌بینی مثبت اشتباه
- **FN** پیش‌بینی منفی اشتباه

نمونه ساختار ماتریس :

	Predicted	
	۰	۱
Actual 0	TN	FP
Actual 1	FN	TP

Confusion Matrix Display

Confusion Matrix Display نمایش گرافیکی ماتریس درهم‌ریختگی است که توسط کتابخانه Scikit-Learn ارائه می‌شود. این نمودار امکان تحلیل سریع تعداد پیش‌بینی‌های صحیح و نادرست مدل را فراهم می‌کند و درک عملکرد مدل را ساده‌تر می‌سازد.

Classification Report

گزارش طبقه‌بندی (Classification Report) خلاصه‌ای از مهم‌ترین معیارهای ارزیابی شامل Precision ، Recall ، F1-Score و Support را برای هر کلاس ارائه می‌دهد. این گزارش دید جامعی از عملکرد مدل در شناسایی کلاس‌های مختلف فراهم می‌کند و یکی از پرکاربردترین ابزارهای ارزیابی مدل‌های طبقه‌بندی است.

نمونه کد:

```
print("Accuracy =", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
confusion_matrix(y_test, y_pred)
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(
    best_knn,
    X_test,
    y_test
)
```

نتایج دقت

جدول زیر نتایج نهایی مدل‌ها را نشان می‌دهد:

نام مدل	دقت یا Accuracy
Logestic Regression	۰,۶۱
KNN	۰,۶۴
SVC	۰,۶۶
Random Forest Classifier	۰,۸۲
Ada Boost Classifier	۰,۷۳
Gradient Booszt Classifier	۰,۸

نتیجه گیری

در این پروژه کیفیت آب آشامیدنی با استفاده از الگوریتم‌های مختلف یادگیری ماشین پیش‌بینی شد. ابتدا داده‌ها پاک‌سازی شده و مقادیر گم‌شده برطرف گردید. سپس تحلیل همبستگی، تشخیص داده‌های پرت و خوشه‌بندی انجام شد. در ادامه داده‌ها استانداردسازی شده و مدل‌های مختلف طبقه‌بندی آموزش داده شدند.

پس از تنظیم پارامترها با استفاده از GridSearchCV، عملکرد مدل‌ها مورد مقایسه قرار گرفت. نتایج نشان داد که مدل Random Forest یا بهترین مدل به دست آمده، عملکرد بهتری نسبت به سایر الگوریتم‌ها داشته و بالاترین دقت را در پیش‌بینی قابلیت آشامیدن آب ارائه کرده است. بنابراین می‌توان نتیجه گرفت که استفاده از روش‌های یادگیری ماشین می‌تواند ابزاری مناسب برای ارزیابی کیفیت آب و کمک به تصمیم‌گیری در حوزه منابع آب باشد.